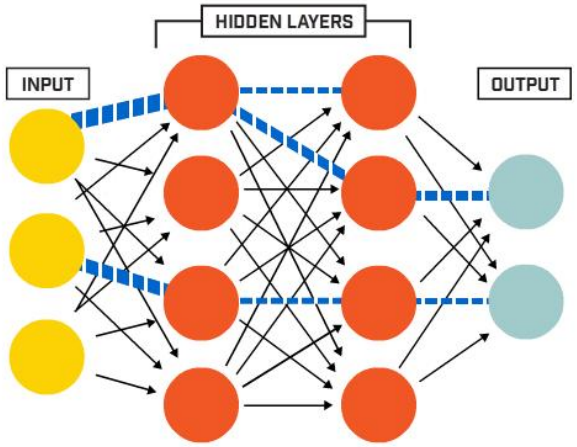
首都师范大学
CAPITAL NORMAL UNIVERSITY



第十二课
神经网络I

授课教师：潘贤丽

对比维度	符号主义（Symbolism）	连接主义（Connectionism）	行为主义（Actionism）
别称	逻辑主义、心理学派	仿生学派、神经网络学派	进化主义、控制论学派
核心思想	智能的本质是**符号的逻辑推理**，通过人为定义规则、符号和逻辑体系，让机器模拟人类的抽象思维	智能的本质是**神经元之间的连接与信息传递**，通过构建仿生神经网络，让机器从数据中自主学习特征	智能的本质是**感知 - 行动的交互反馈**，通过与环境的持续互动、试错和进化，让机器逐步适应环境
理论基础	数理逻辑、知识工程、形式化推理	神经科学、脑科学、统计学习、优化理论	控制论、进化论、强化学习
智能来源	**人工植入**：由人类专家编写规则和知识图谱，机器被动执行推理	**自主学习**：从海量数据中学习神经元连接权重，自动提取特征和规律	**环境交互**：通过与环境的反馈（奖励 / 惩罚）调整行为策略，逐步进化
核心技术	专家系统、谓词逻辑、语义网络、定理证明器	神经网络、深度学习（CNN/RNN/Transformer）、SOM、BP 算法	强化学习、遗传算法、行为机器人、进化策略

大纲

基础知识

神经元　MLP　BP算法

拓展知识

CNN　RNN　Transformer

图像　序列　注意力机制

VAE　GAN　Diffusion

生成模型系列

大纲

基础知识

- ✓ 神经元
- MLP
- BP算法

拓展知识

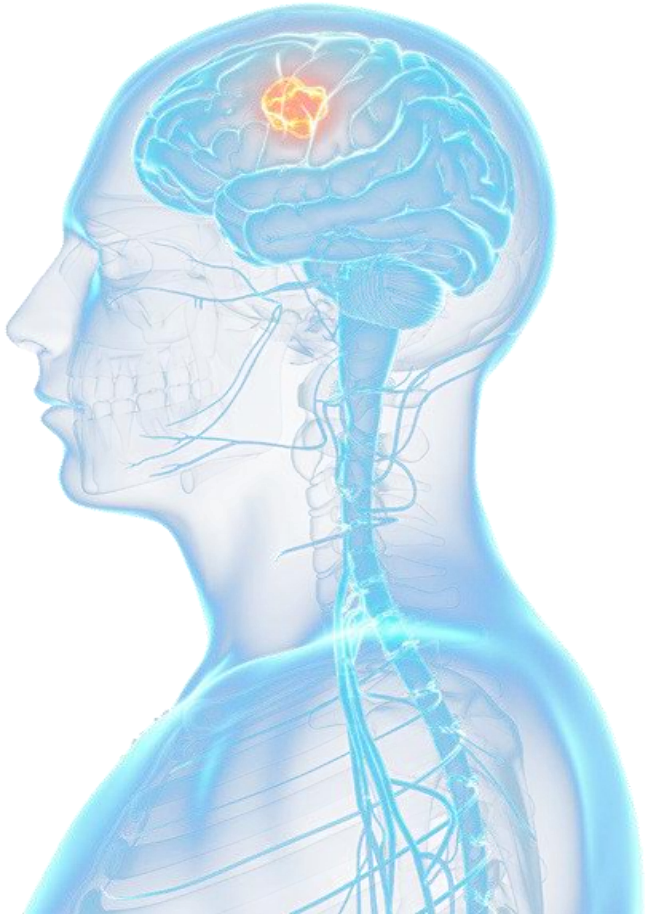
- CNN
- RNN
- Transformer

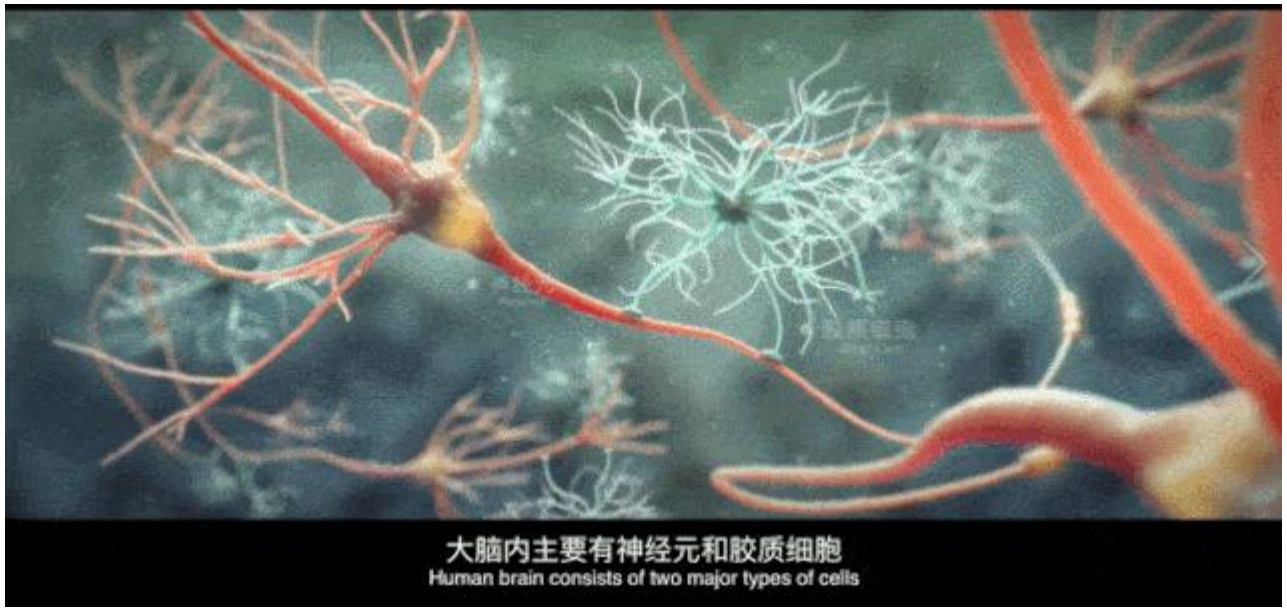
图像　序列　注意力机制

- VAE
- GAN
- Diffusion

生成模型系列

神经元

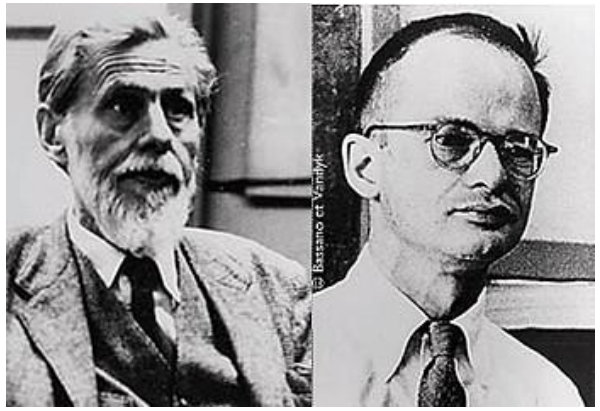




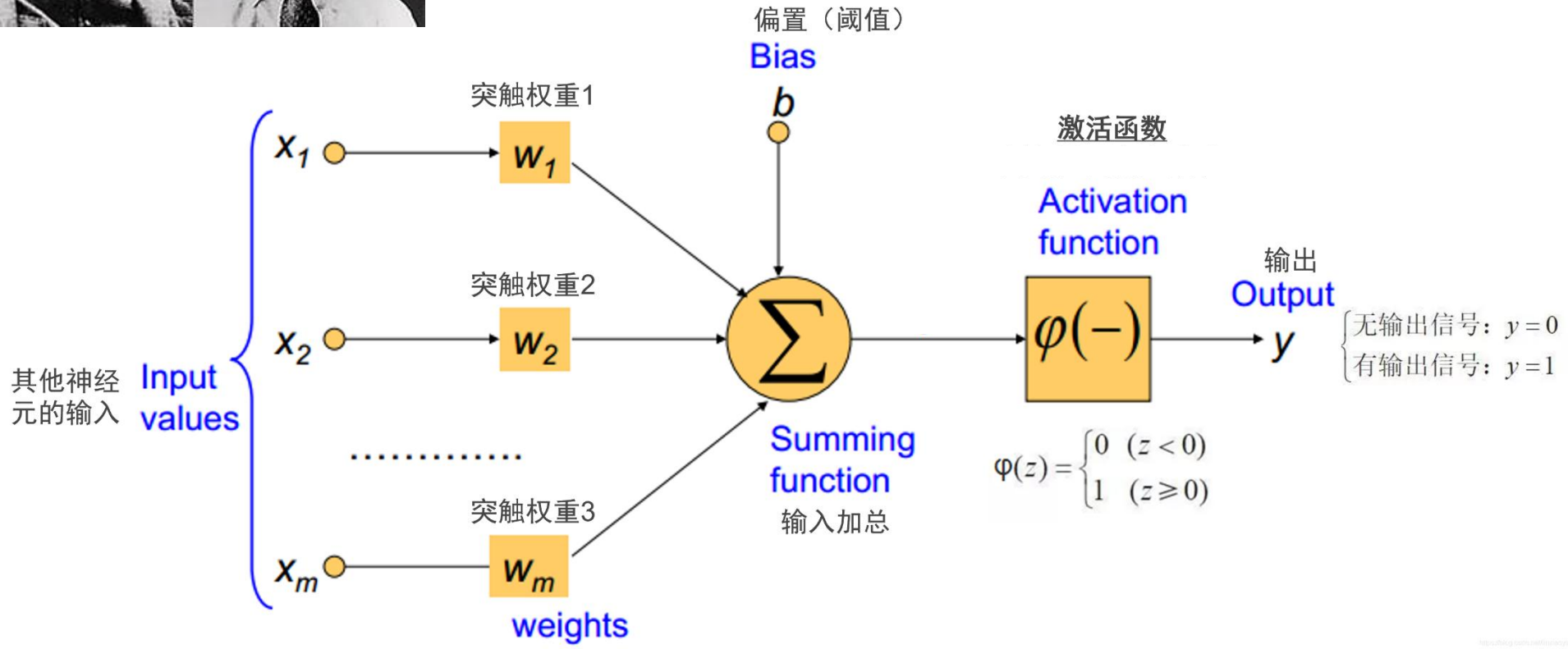
生物神经网络：神经元——突触——神经元

神经元的电位超过一个**阈值**，会被**激活**，向其它神经元发送化学物质，犹如涟漪，**一层接着一层传播。**

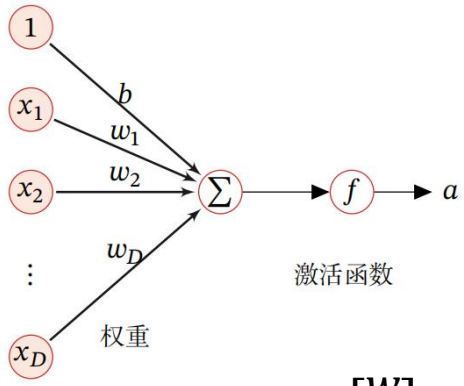
神经元



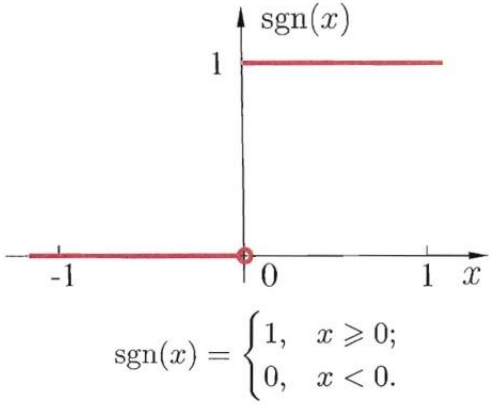
1943年，心理学家McCulloch和逻辑学家Pitts建立神经网络的数学模型：**M-P神经元模型**



神经元



$$\mathrm{w} = \begin{bmatrix} w \\ b \end{bmatrix} \qquad x = \begin{bmatrix} x \\ 1 \end{bmatrix}$$



$$\mathrm{sgn}(x) = \begin{cases} 1, & x \geqslant 0; \\ 0, & x < 0. \end{cases}$$

(a) 阶跃函数

M-P神经元模型

$$f(x) = \mathrm{sgn}(w^T x)$$

单层感知机

错误驱动的在线学习算法：每次用分错的样本更新权重

$$\boldsymbol{w} \leftarrow \boldsymbol{w} + y\boldsymbol{x}$$

算法 3.1　两类感知器的参数学习算法

输入: 训练集 $\mathcal{D} = \{(\boldsymbol{x}^{(n)}, y^{(n)})\}_{n=1}^{N}$，最大迭代次数 T

初始化：$\boldsymbol{w}_0 \leftarrow 0, k \leftarrow 0, t \leftarrow 0$；
repeat
　对训练集 $\mathcal{D}$ 中的样本随机排序；
　for $n = 1 \cdots N$ **do**
　　选取一个样本 $(\boldsymbol{x}^{(n)}, y^{(n)})$；
　　if $\boldsymbol{w}_k^{\mathrm{T}}(y^{(n)}\boldsymbol{x}^{(n)}) \leq 0$ **then**
　　　$\boldsymbol{w}_{k+1} \leftarrow \boldsymbol{w}_k + y^{(n)}\boldsymbol{x}^{(n)}$；
　　　$k \leftarrow k + 1$；
　　end
　　$t \leftarrow t + 1$；
　　if $t = T$ **then** break；
　end
until $t = T$；
输出: $\boldsymbol{w}_k$

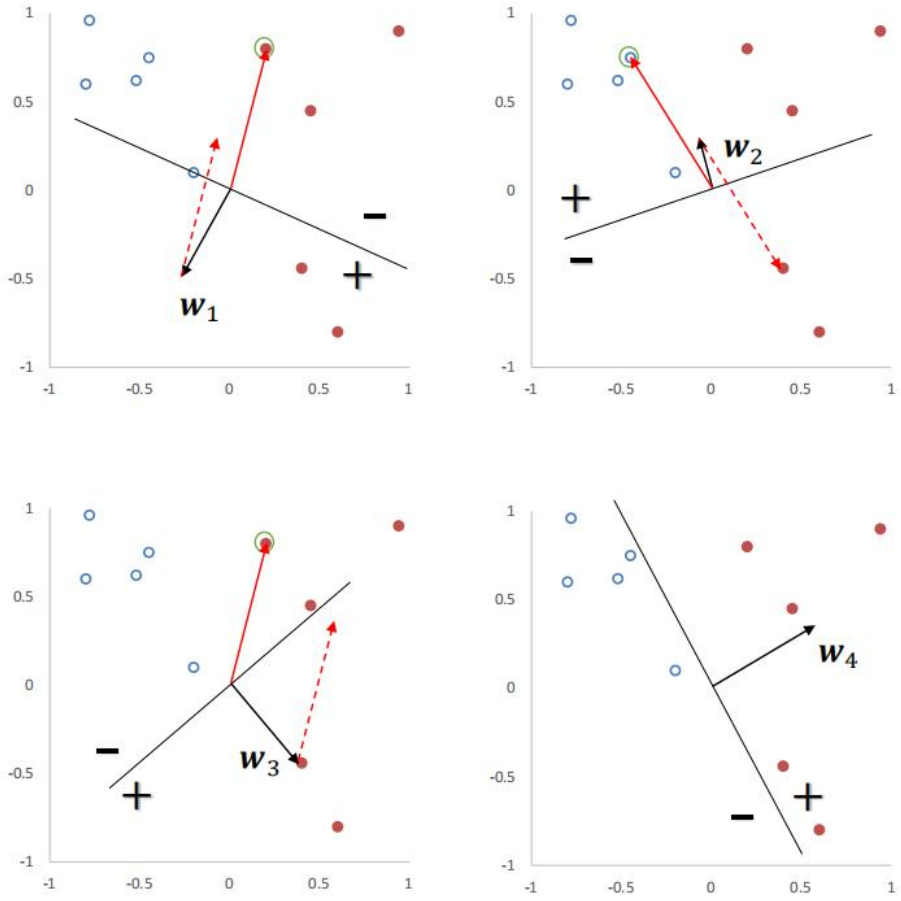


图 3.5　感知器参数学习的更新过程

只能处理线性可分二分类问题

大纲

基础知识

神经元 | MLP | BP算法

拓展知识

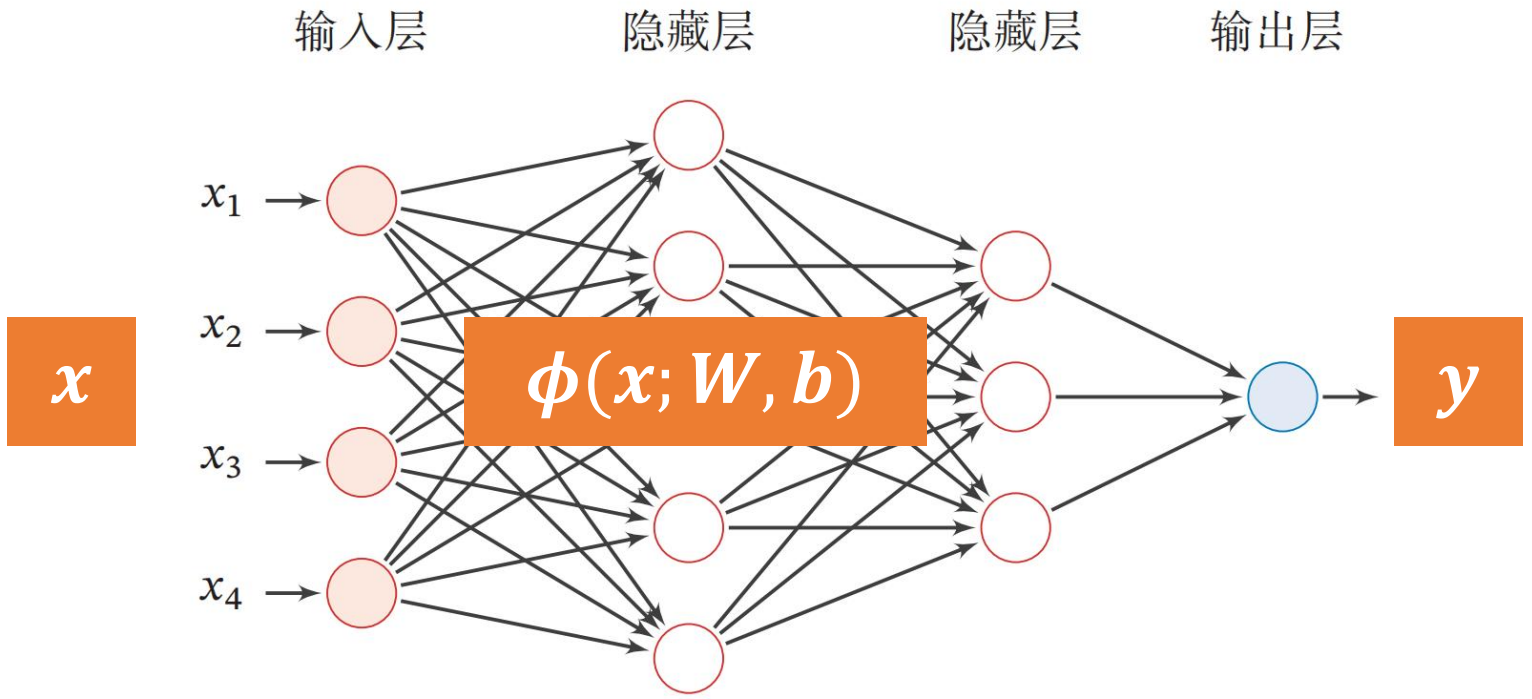
CNN | RNN | Transformer

图像 | 序列 | 注意力机制

VAE | GAN | Diffusion

生成模型系列

多层感知机(MLP)

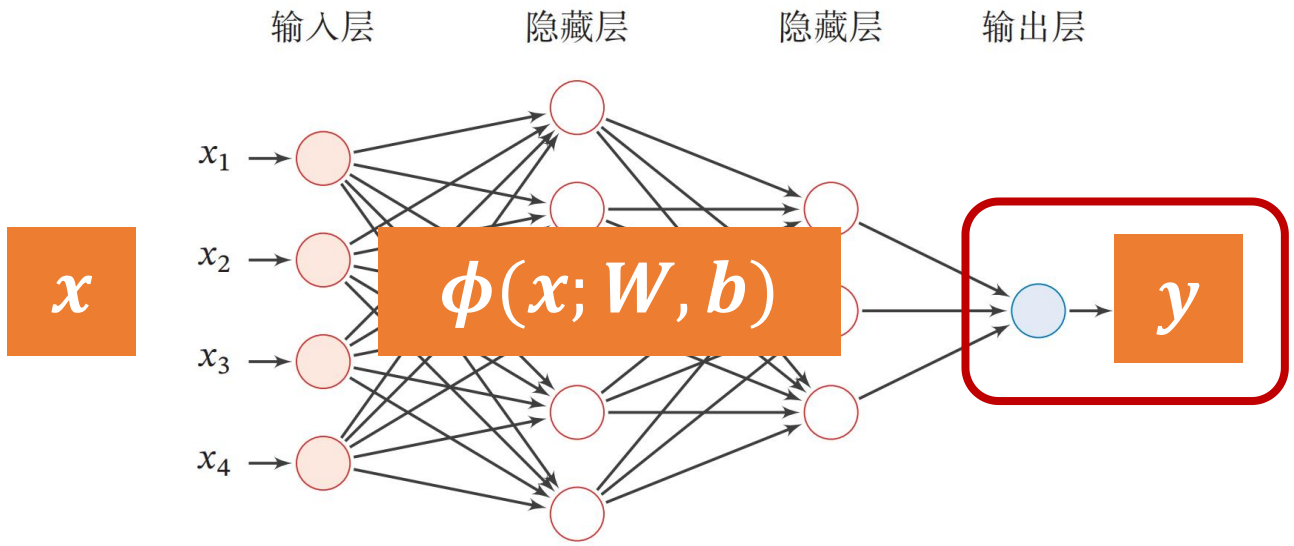


$$a^{(0)} = x$$

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)},$$
$$a^{(l)} = f_l(z^{(l)}).$$

$$x = a^{(0)} \to z^{(1)} \to a^{(1)} \to z^{(2)} \to \cdots \to a^{(L-1)} \to z^{(L)} \to a^{(L)} = \phi(x; W, b)$$

多层感知机(MLP)

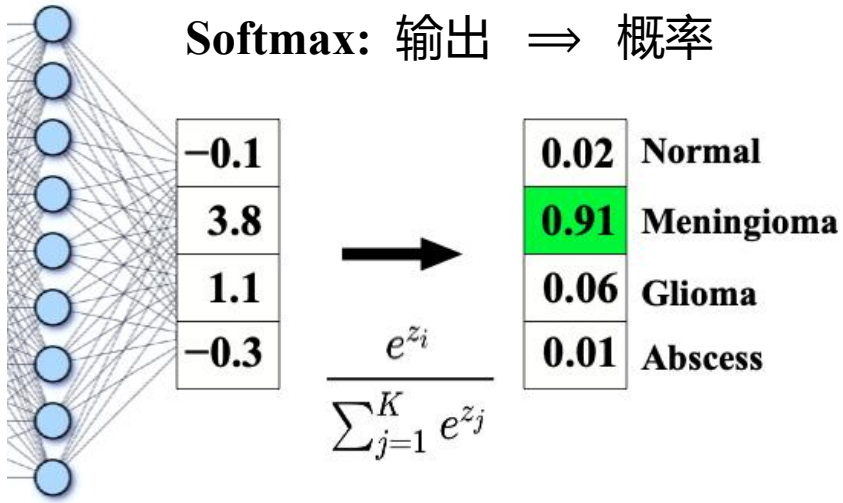


回归

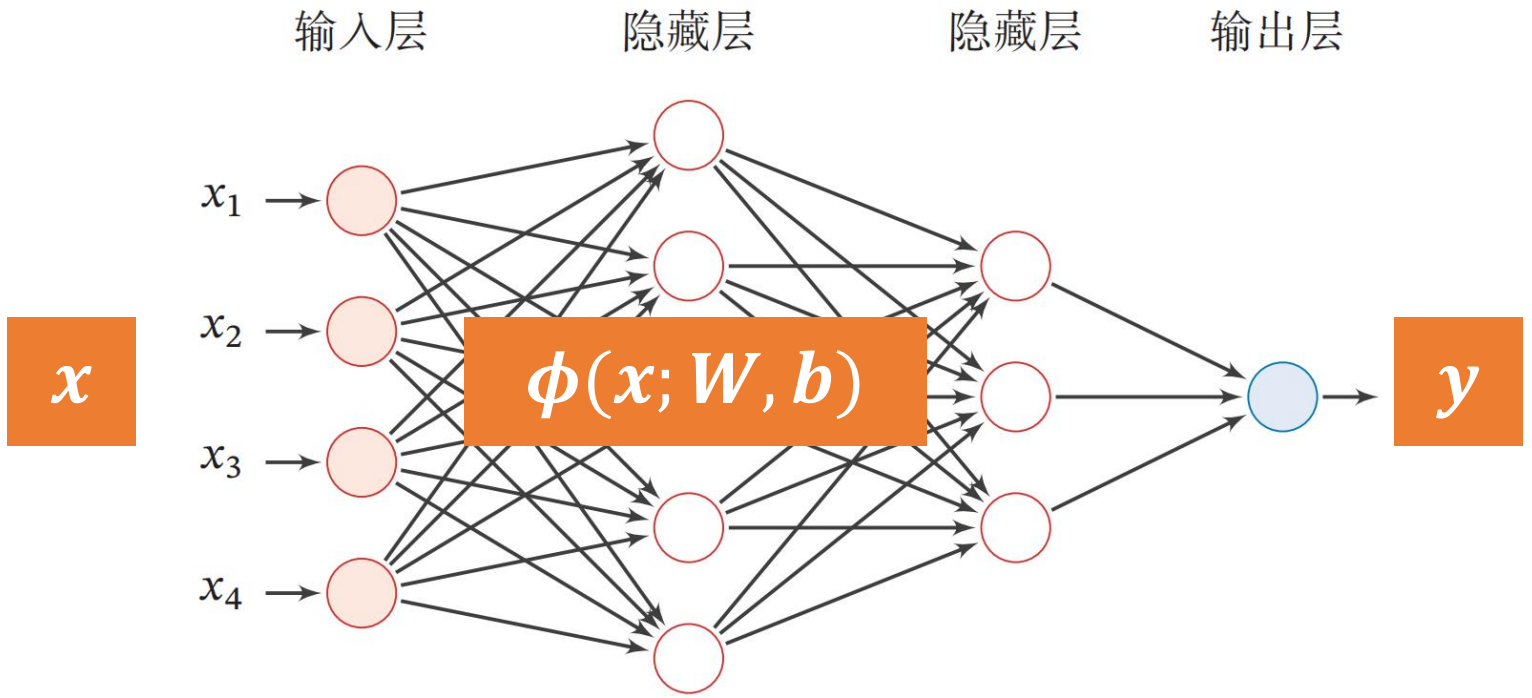
$y \in R$

多分类

$y \in R^{K*1}$



多层感知机(MLP)

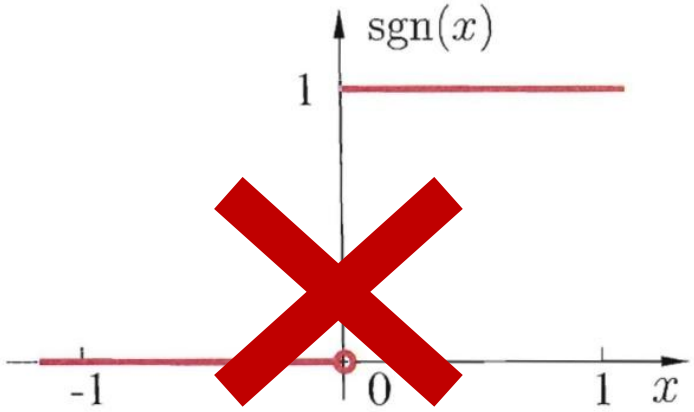


$$\boldsymbol{a}^{(0)} = \boldsymbol{x}$$

$$\begin{aligned} \boldsymbol{z}^{(l)} &= \boldsymbol{W}^{(l)}\boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)}, \\ \boldsymbol{a}^{(l)} &= f_l(\boldsymbol{z}^{(l)}). \end{aligned}$$

$$\boldsymbol{x} = \boldsymbol{a}^{(0)} \to \boldsymbol{z}^{(1)} \to \boldsymbol{a}^{(1)} \to \boldsymbol{z}^{(2)} \to \cdots \to \boldsymbol{a}^{(L-1)} \to \boldsymbol{z}^{(L)} \to \boldsymbol{a}^{(L)} = \phi(\boldsymbol{x}; \boldsymbol{W}, \boldsymbol{b})$$

多层感知机(MLP)



$$\operatorname{sgn}(x) = \begin{cases} 1, & x \geqslant 0; \\ 0, & x < 0. \end{cases}$$

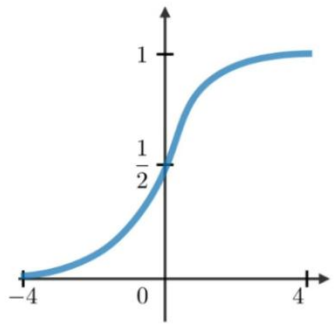
(a) 阶跃函数

为什么激活函数都是非线性的?

常见激活函数

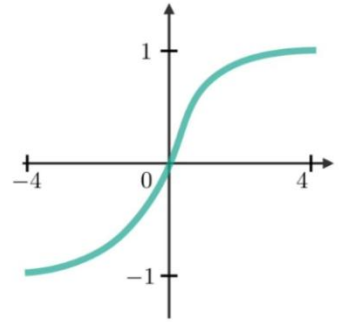
Sigmoid

$$g(z) = \frac{1}{1+e^{-z}}$$



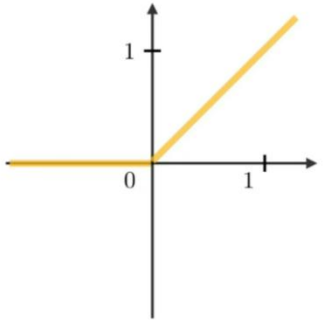
Tanh

$$g(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$$



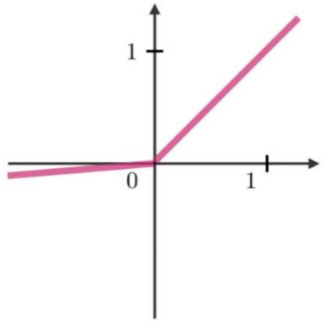
RELU

$$g(z) = \max(0, z)$$



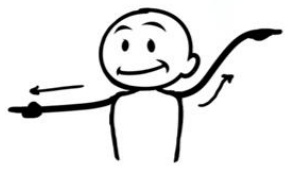
Leaky ReLU

$$g(z) = \max(\epsilon z, z)$$
with $\epsilon \ll 1$



多层感知机(MLP)

Sigmoid



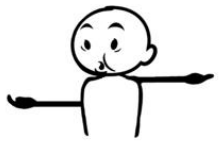
$$y = \frac{1}{1+e^{-x}}$$

Tanh



$$y = \tanh(x)$$

Step Function



$$y = \begin{cases} 0, & x<n \\ 1, & x \geq n \end{cases}$$

Softplus



$$y = \ln(1+e^{x})$$

ReLU



$$y = \begin{cases} 0, & x<0 \\ x, & x \geq 0 \end{cases}$$

Softsign



$$y = \frac{x}{(1+|x|)}$$

ELU



$$y = \begin{cases} \alpha(e^{x}-1), & x<0 \\ x, & x \geq 0 \end{cases}$$

Log of Sigmoid



$$y = \ln\left(\frac{1}{1+e^{-x}}\right)$$

Swish



$$y = \frac{x}{1+e^{-x}}$$

Sinc



$$y = \frac{\sin(x)}{x}$$

Leaky ReLU



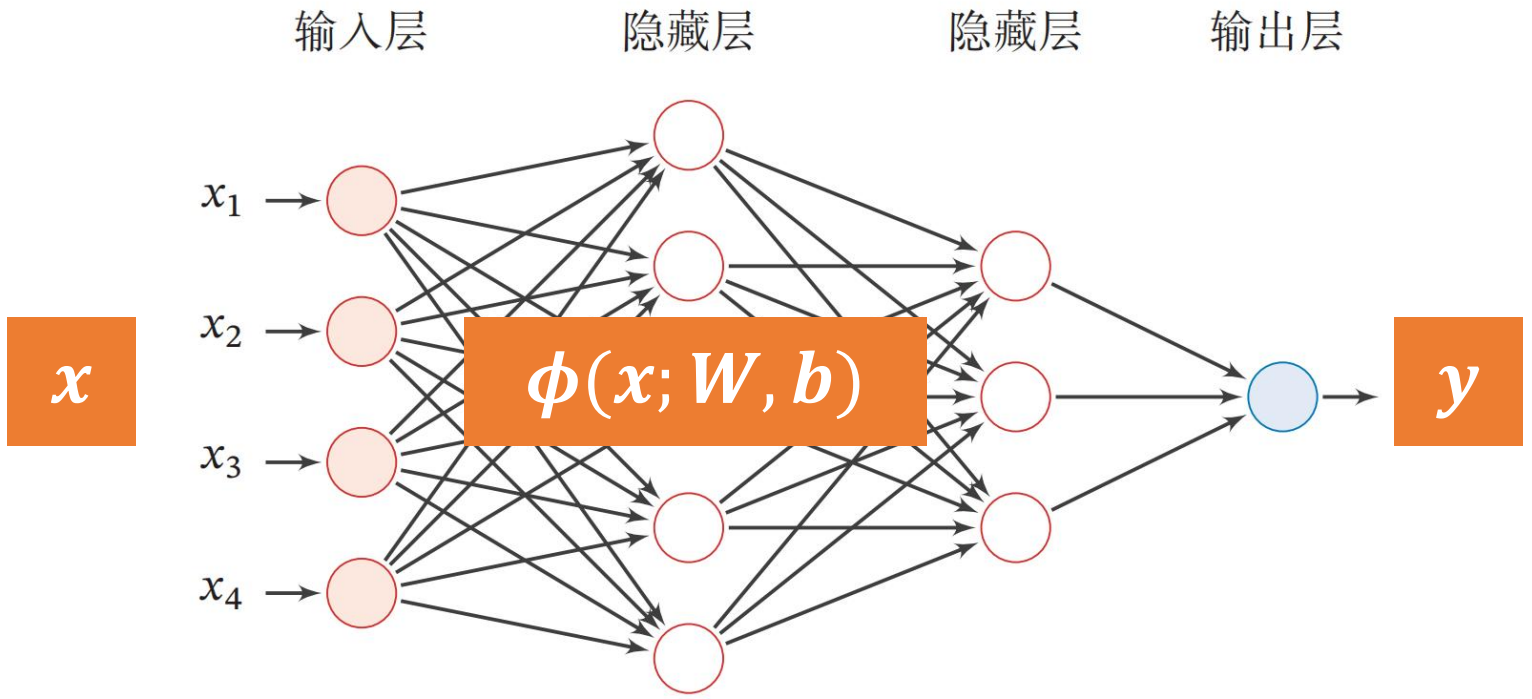
$$y = \max(0.1x, x)$$

Mish



$$y = x(\tanh(\text{softplus}(x)))$$

多层感知机(MLP)

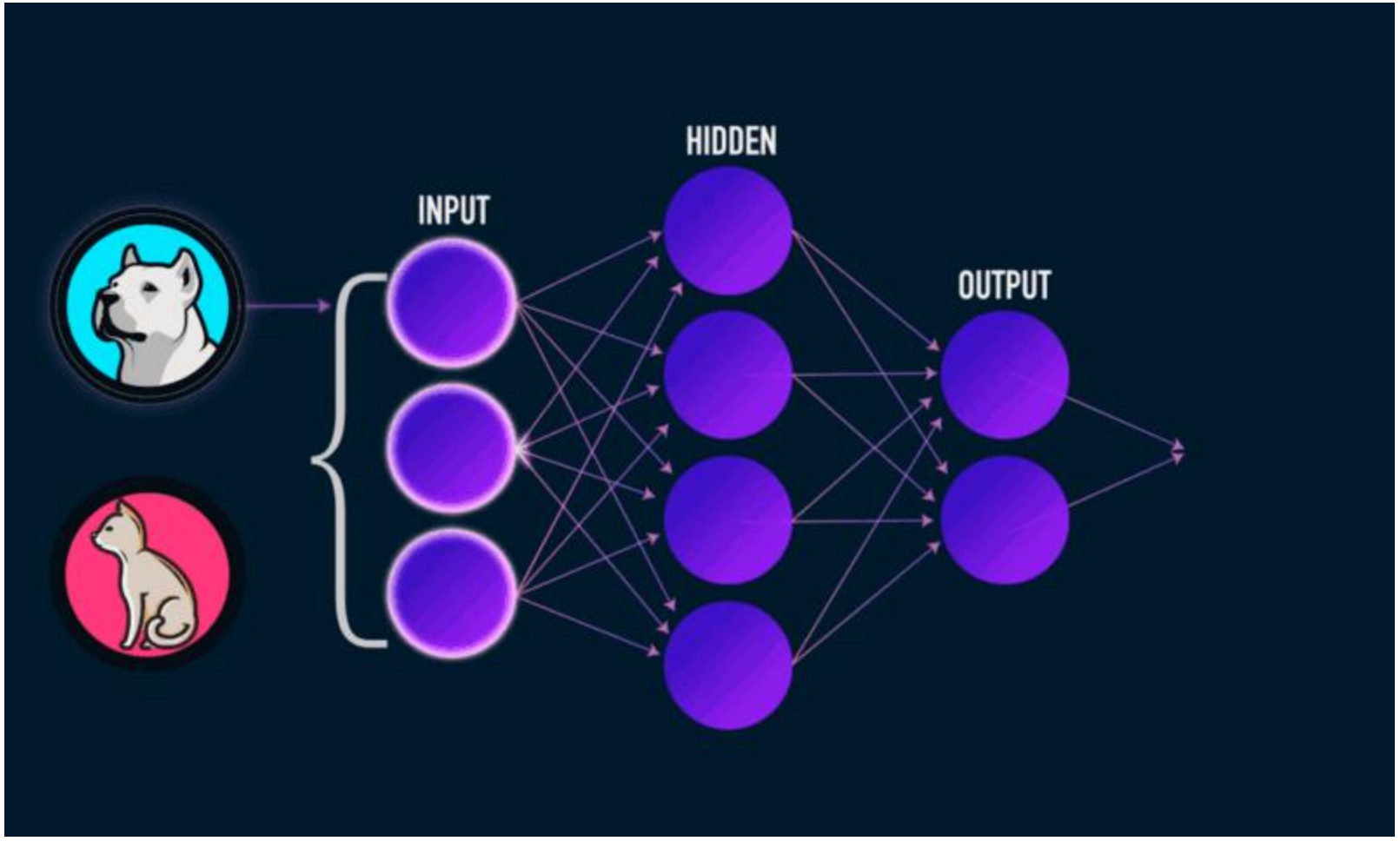


输出端怎么设定?

$\phi(x; W, b)$长什么样?

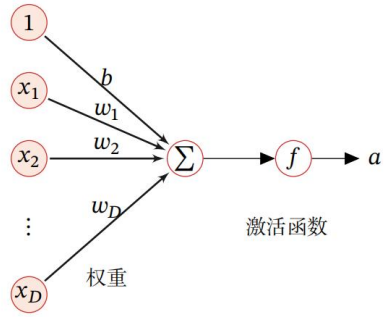
激活函数有哪些?

多层感知机(MLP)



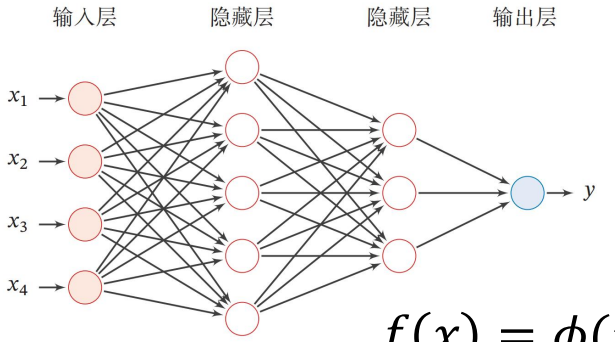
多层感知机(MLP)

单层感知机



$$f(x) = \text{sgn}(w^T x + b)$$

多层感知机



$$f(x) = \phi(x; W, b)$$

定理 4.1 – 通用近似定理（Universal Approximation Theorem）[Cybenko, 1989; Hornik et al., 1989]：令 $\phi(\cdot)$ 是一个非常数、有界、单调递增的连续函数，$\mathcal{I}_D$ 是一个 D 维的单位超立方体 $[0,1]^D$，$C(\mathcal{I}_D)$ 是定义在 $\mathcal{I}_D$ 上的连续函数集合. 对于任意给定的一个函数 $f \in C(\mathcal{I}_D)$，存在一个整数 M，和一组实数 $v_m, b_m \in \mathbb{R}$ 以及实数向量 $\boldsymbol{w}_m \in \mathbb{R}^D$，$m = 1, \cdots, M$，以至于我们可以定义函数

$$F(\boldsymbol{x}) = \sum_{m=1}^{M} v_m \phi(\boldsymbol{w}_m^{\mathrm{T}} \boldsymbol{x} + b_m), \tag{4.37}$$

作为函数 f 的近似实现，即

$$|F(\boldsymbol{x}) - f(\boldsymbol{x})| < \epsilon, \forall \boldsymbol{x} \in \mathcal{I}_D, \tag{4.38}$$

其中 $\epsilon > 0$ 是一个很小的正数.

怎么求解?

基础知识

神经元　MLP　BP算法

拓展知识

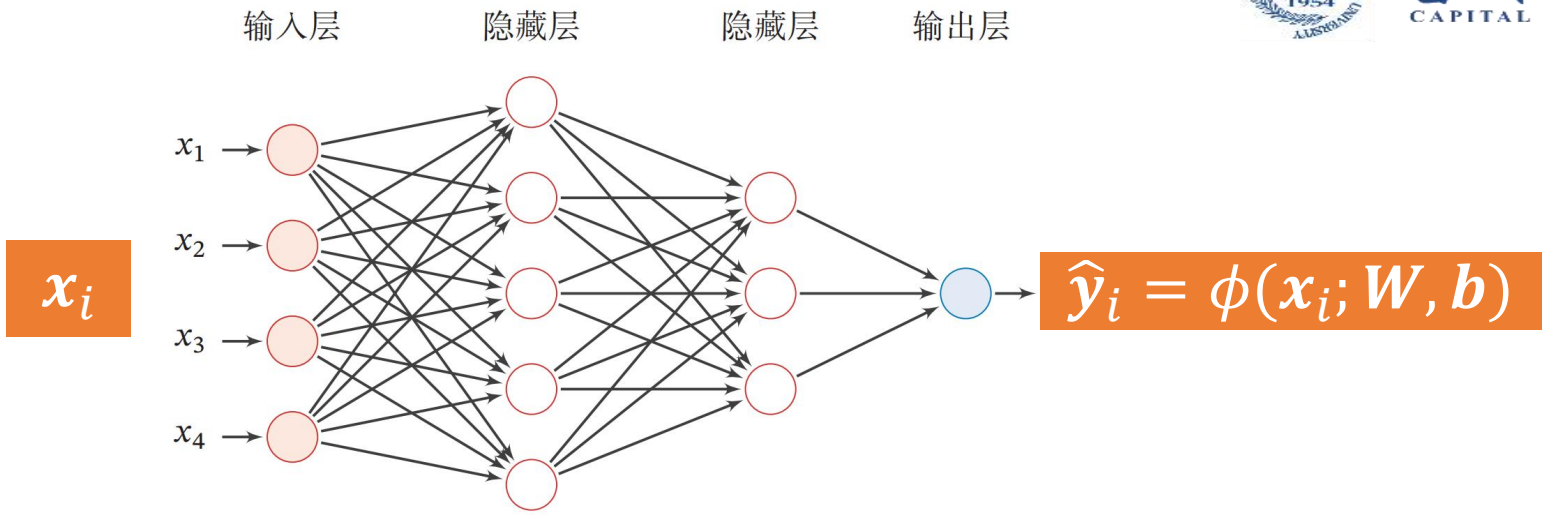
CNN　RNN　Transformer

图像　序列　注意力机制

VAE　GAN　Diffusion

生成模型系列

BP算法



$$x = a^{(0)} \to z^{(1)} \to a^{(1)} \to z^{(2)} \to \cdots \to a^{(L-1)} \to z^{(L)} \to a^{(L)} = \phi(x; W, b)$$

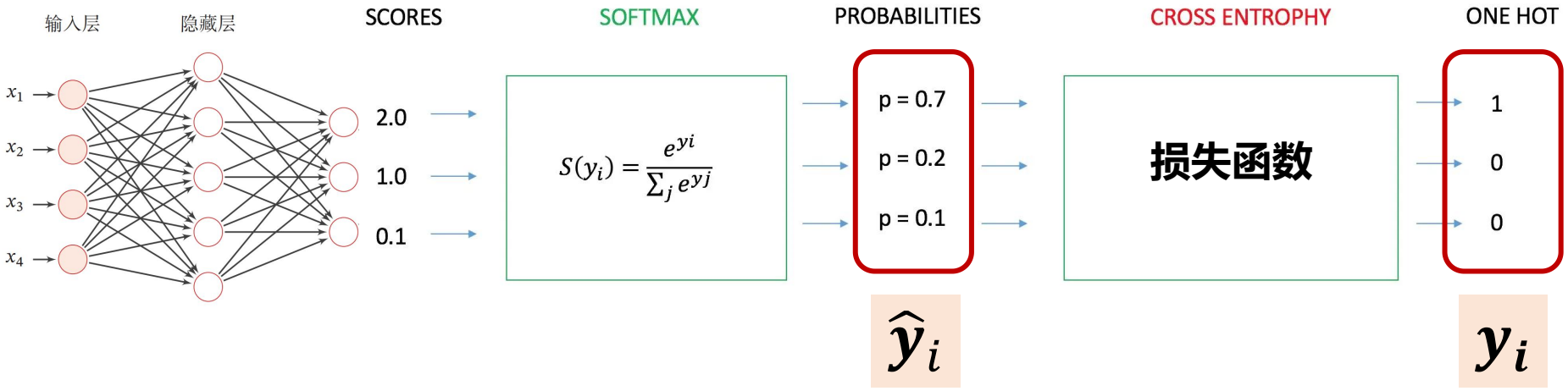
待学习参数

$W^{(1)}, b^{(1)}$
$W^{(2)}, b^{(2)}$
……
$W^{(L)}, b^{(L)}$

设定损失函数，找到使损失函数最小的参数



BP算法



分类损失函数：交叉熵

$$min - \sum_{i=1}^{n} y_i^T log(\hat{y}_i)$$

取值范围： $[0, +\infty)$

BP算法

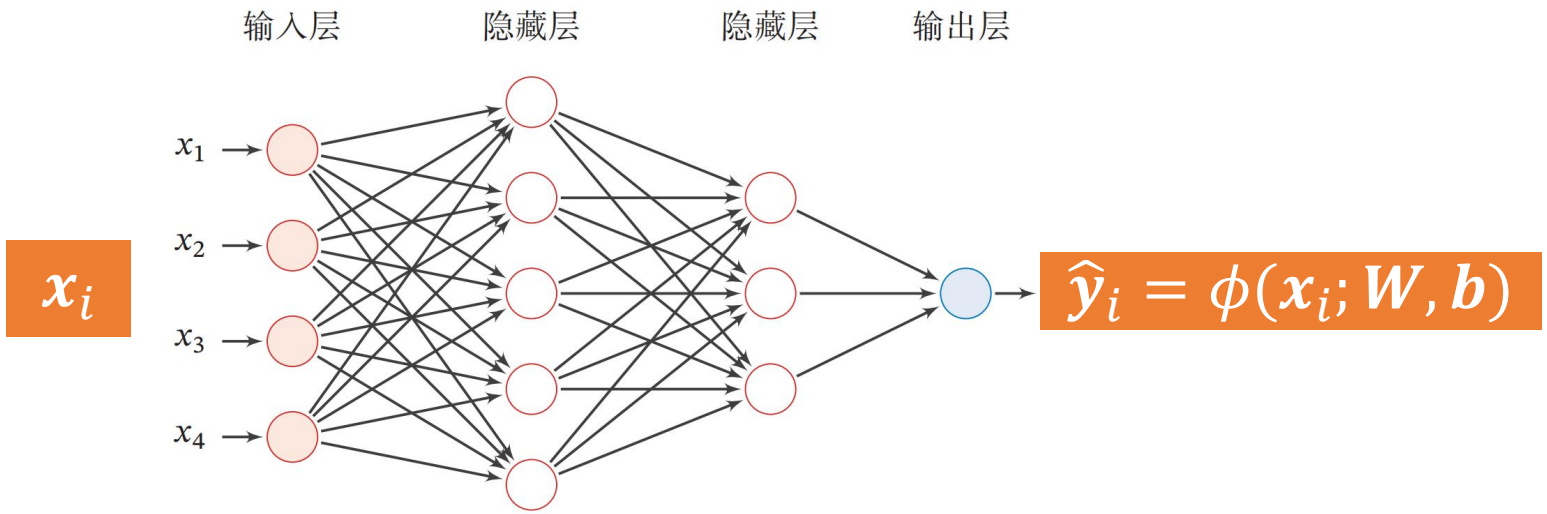






	Prediction	Target	Loss	Prediction	Target	Loss	Prediction	Target	Loss
Cat	**0.3**	**1**	**1.2**	0.1	0	0	0.5	0	0
Car	0.5	0	0	**0.8**	**1**	**0.22**	0.45	0	0
Rhino	0.2	0	0	0.1	0	0	**0.05**	**1**	**2.99**

BP算法



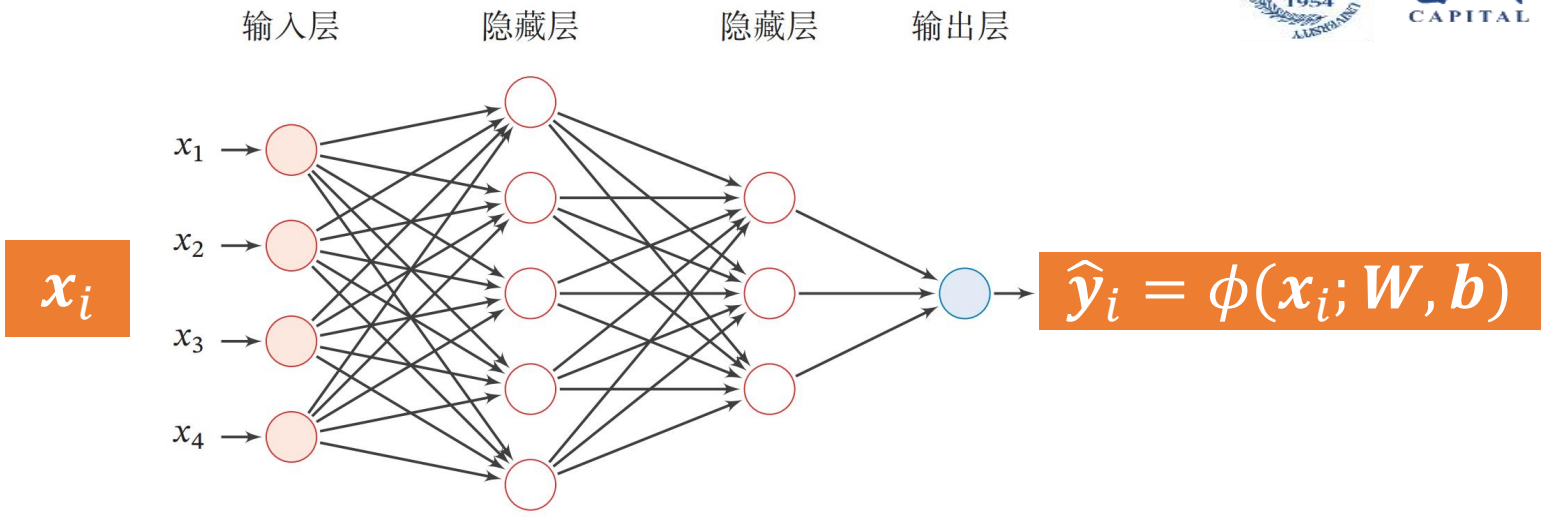
$$x = a^{(0)} \to z^{(1)} \to a^{(1)} \to z^{(2)} \to \cdots \to a^{(L-1)} \to z^{(L)} \to a^{(L)} = \phi(x; W, b)$$

回归损失函数：

MSE

$$min \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

BP算法



$$x = a^{(0)} \to z^{(1)} \to a^{(1)} \to z^{(2)} \to \cdots \to a^{(L-1)} \to z^{(L)} \to a^{(L)} = \phi(x; W, b)$$

待学习参数

$W^{(1)}, b^{(1)}$
$W^{(2)}, b^{(2)}$
……
$W^{(L)}, b^{(L)}$

设定损失函数，找到使损失函数最小的参数



$$min\ L(\boldsymbol{W},\boldsymbol{b}) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$minL(\boldsymbol{W},\boldsymbol{b}) = -\sum_{i=1}^{n}\boldsymbol{y}_i^T\boldsymbol{log}(\hat{\boldsymbol{y}}_i)$$

求解

$$\boldsymbol{W}^{(1)},\ \boldsymbol{b}^{(1)}$$
$$\boldsymbol{W}^{(2)},\ \boldsymbol{b}^{(2)}$$
$$\cdots\cdots$$
$$\boldsymbol{W}^{(L)},\ \boldsymbol{b}^{(L)}$$

$\hat{\boldsymbol{y}}_i = \boldsymbol{\phi}(\boldsymbol{x}_i; \boldsymbol{W}, \boldsymbol{b})$：多层复合函数

BP算法 = 梯度下降 + 链式求导法则

$$\boldsymbol{W}_{\boldsymbol{k+1}}^{(\boldsymbol{l})} = \boldsymbol{W}_{\boldsymbol{k}}^{(\boldsymbol{l})} - \boldsymbol{\eta}\frac{\boldsymbol{\partial L}}{\boldsymbol{\partial W}^{(\boldsymbol{l})}}\Big|_{\boldsymbol{W}^{(\boldsymbol{l})}=\boldsymbol{W}_{\boldsymbol{k}}^{(\boldsymbol{l})}}$$

$$min\ L(\boldsymbol{W},\boldsymbol{b}) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$minL(\boldsymbol{W},\boldsymbol{b}) = -\sum_{i=1}^{n}\boldsymbol{y}_i^T\boldsymbol{log}(\hat{\boldsymbol{y}}_i)$$

求解

$$\boldsymbol{W}^{(1)},\ \boldsymbol{b}^{(1)}$$
$$\boldsymbol{W}^{(2)},\ \boldsymbol{b}^{(2)}$$
$$\cdots\cdots$$
$$\boldsymbol{W}^{(L)},\ \boldsymbol{b}^{(L)}$$

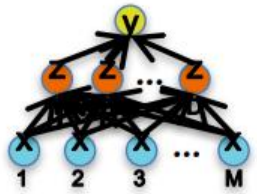
$\hat{\boldsymbol{y}}_i = \boldsymbol{\phi}(\boldsymbol{x}_i; \boldsymbol{W}, \boldsymbol{b})$：多层复合函数

BP算法 = 梯度下降 + 链式求导法则

$$\boldsymbol{W}_{\boldsymbol{k+1}}^{(\boldsymbol{l})} = \boldsymbol{W}_{\boldsymbol{k}}^{(\boldsymbol{l})} - \boldsymbol{\eta}\frac{\boldsymbol{\partial L}}{\boldsymbol{\partial W}^{(\boldsymbol{l})}}\Big|_{\boldsymbol{W}^{(\boldsymbol{l})}=\boldsymbol{W}_{\boldsymbol{k}}^{(\boldsymbol{l})}}$$

链式求导法则

两层神经网络



前向

$$J = y^* \log y + (1 - y^*) \log(1 - y)$$

$$y = \frac{1}{1 + \exp(-b)}$$

$$b = \sum_{j=0}^{D} \beta_j z_j$$

参数：α_{ji}、β_j

$$z_j = \frac{1}{1 + \exp(-a_j)}$$

$$a_j = \sum_{i=0}^{M} \alpha_{ji} x_i$$

参数更新： $\beta_j^{(k+1)} = \beta_j^{(k)} - \eta \frac{\partial J}{\partial \beta_j}|_{\beta_j = \beta_j^{(k)}}$

反向

$$\frac{dJ}{dy} = \frac{y^*}{y} + \frac{(1 - y^*)}{y - 1}$$

$$\frac{dJ}{db} = \frac{dJ}{dy}\frac{dy}{db}, \frac{dy}{db} = \frac{\exp(-b)}{(\exp(-b) + 1)^2}$$

$$\frac{dJ}{d\beta_j} = \frac{dJ}{db}\frac{db}{d\beta_j}, \frac{db}{d\beta_j} = z_j$$

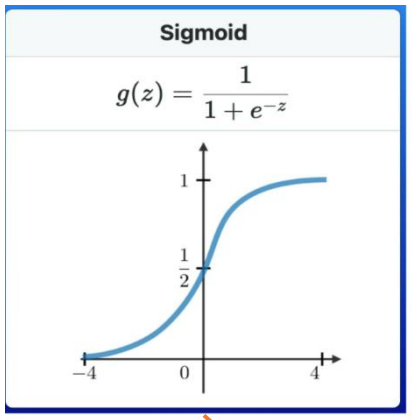
$$\frac{dJ}{dz_j} = \frac{dJ}{db}\frac{db}{dz_j}, \frac{db}{dz_j} = \beta_j$$

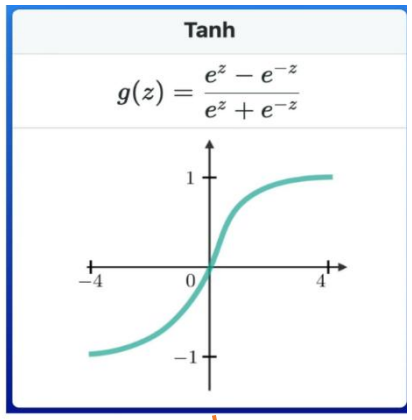
$$\frac{dJ}{da_j} = \frac{dJ}{dz_j}\frac{dz_j}{da_j}, \frac{dz_j}{da_j} = \frac{\exp(-a_j)}{(\exp(-a_j) + 1)^2}$$

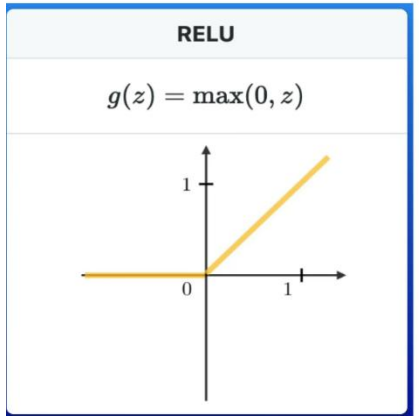
$$\frac{dJ}{d\alpha_{ji}} = \frac{dJ}{da_j}\frac{da_j}{d\alpha_{ji}}, \frac{da_j}{d\alpha_{ji}} = x_i$$

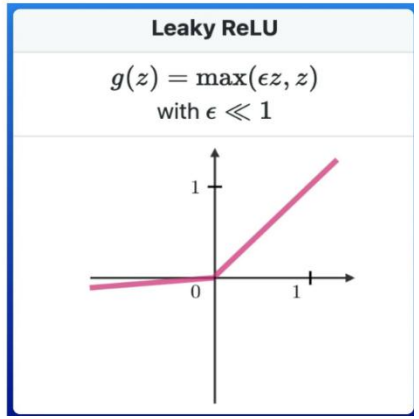
$$\frac{dJ}{dx_i} = \frac{dJ}{da_j}\frac{da_j}{dx_i}, \frac{da_j}{dx_i} = \sum_{j=0}^{D} \alpha_{ji}$$

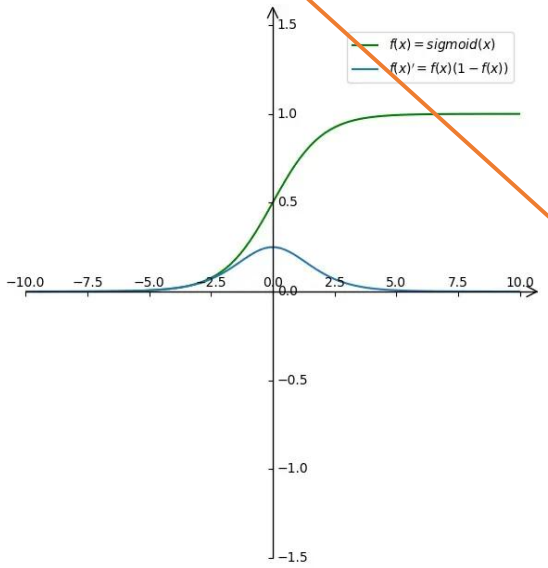
常见激活函数











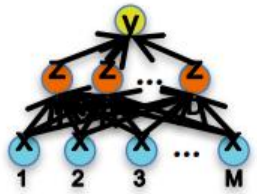
激活函数	函数	导数
Logistic 函数	$f(x) = \frac{1}{1+\exp(-x)}$	$f'(x) = f(x)(1 - f(x))$
Tanh 函数	$f(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$	$f'(x) = 1 - f(x)^2$
ReLU 函数	$f(x) = \max(0, x)$	$f'(x) = I(x > 0)$

梯度消失

死亡神经元

链式求导法则

两层神经网络



前向

$$J = y^* \log y + (1 - y^*) \log(1 - y)$$

$$y = \frac{1}{1 + \exp(-b)}$$

Sigmoid：

- 导数取值范围：$[0,\ 0.25]$
- 导数连乘

$$z_j = \frac{1}{1 + \exp(-a_j)}$$

$$a_j = \sum_{i=0}^{M} \alpha_{ji} x_i$$

反向

$$\frac{dJ}{dy} = \frac{y^*}{y} + \frac{(1 - y^*)}{y - 1}$$

$$\frac{dJ}{db} = \frac{dJ}{dy}\frac{dy}{db}, \boxed{\frac{dy}{db} = \frac{\exp(-b)}{(\exp(-b) + 1)^2}}$$

$$\frac{dJ}{d\beta_j} = \frac{dJ}{db}\frac{db}{d\beta_j}, \frac{db}{d\beta_j} = z_j$$

$$\frac{dJ}{dz_j} = \frac{dJ}{db}\frac{db}{dz_j}, \frac{db}{dz_j} = \beta_j$$

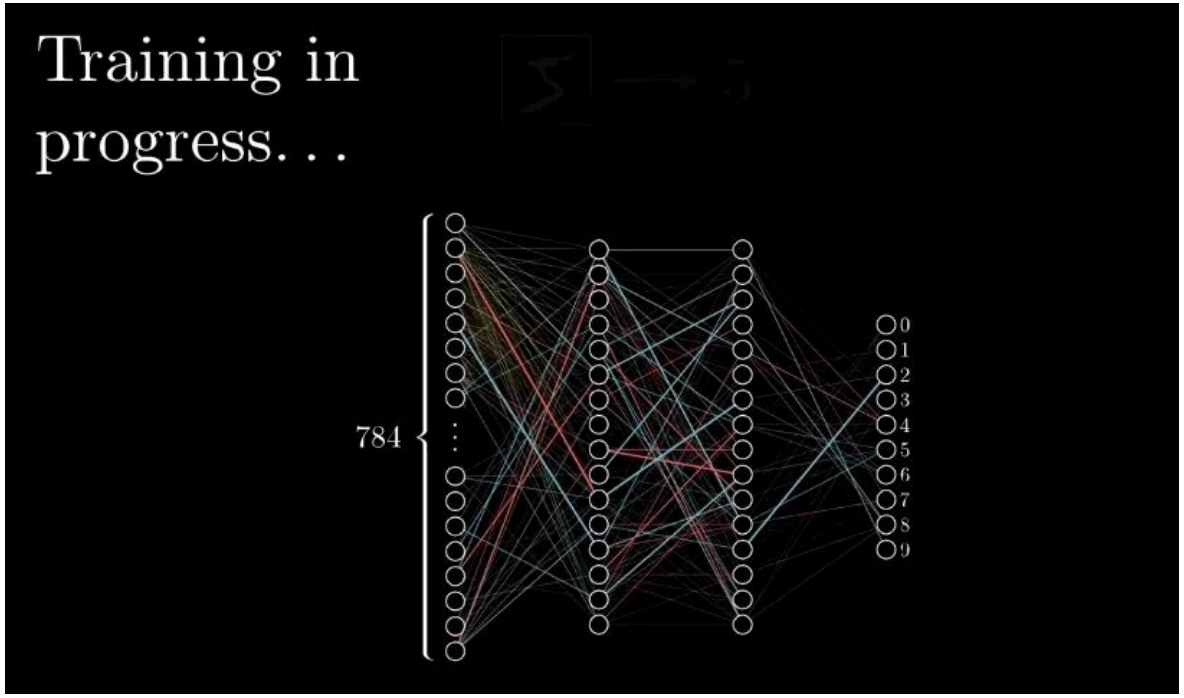
$$\frac{dJ}{da_j} = \frac{dJ}{dz_j}\frac{dz_j}{da_j}, \boxed{\frac{dz_j}{da_j} = \frac{\exp(-a_j)}{(\exp(-a_j) + 1)^2}}$$

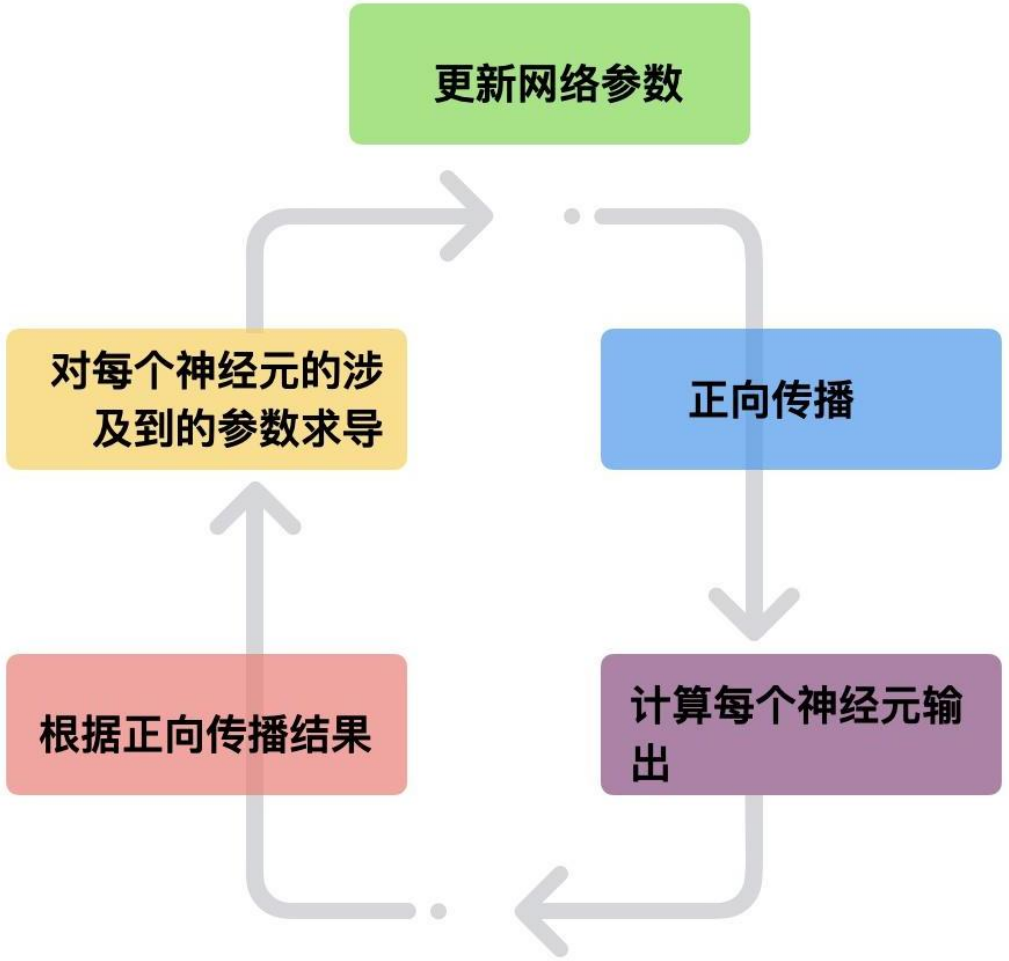
$$\frac{dJ}{d\alpha_{ji}} = \frac{dJ}{da_j}\frac{da_j}{d\alpha_{ji}}, \frac{da_j}{d\alpha_{ji}} = x_i$$

$$\frac{dJ}{dx_i} = \frac{dJ}{da_j}\frac{da_j}{dx_i}, \frac{da_j}{dx_i} = \sum_{j=0}^{D} \alpha_{ji}$$

参数更新： $\beta_j^{(k+1)} = \beta_j^{(k)} - \eta \frac{\partial J}{\partial \beta_j}\Big|_{\beta_j = \beta_j^{(k)}}$

BP算法


Training in progress. . .
784
0
1
2
3
4
5
6
7
8
9


更新网络参数
正向传播
计算每个神经元输出
根据正向传播结果
对每个神经元的涉及到的参数求导

mini-batch

epoch

iteration

假设训练集有 1000 个样本，batch size = 100。

那么 1 个 epoch 内会发生：

- 第 1 次：看前 100 个样本，更新一次
- 第 2 次：看第 101–200 个样本，更新一次
- …
- 第 10 次：看最后 100 个样本，更新一次

这 10 次更新做完，叫 **1 个 epoch**。

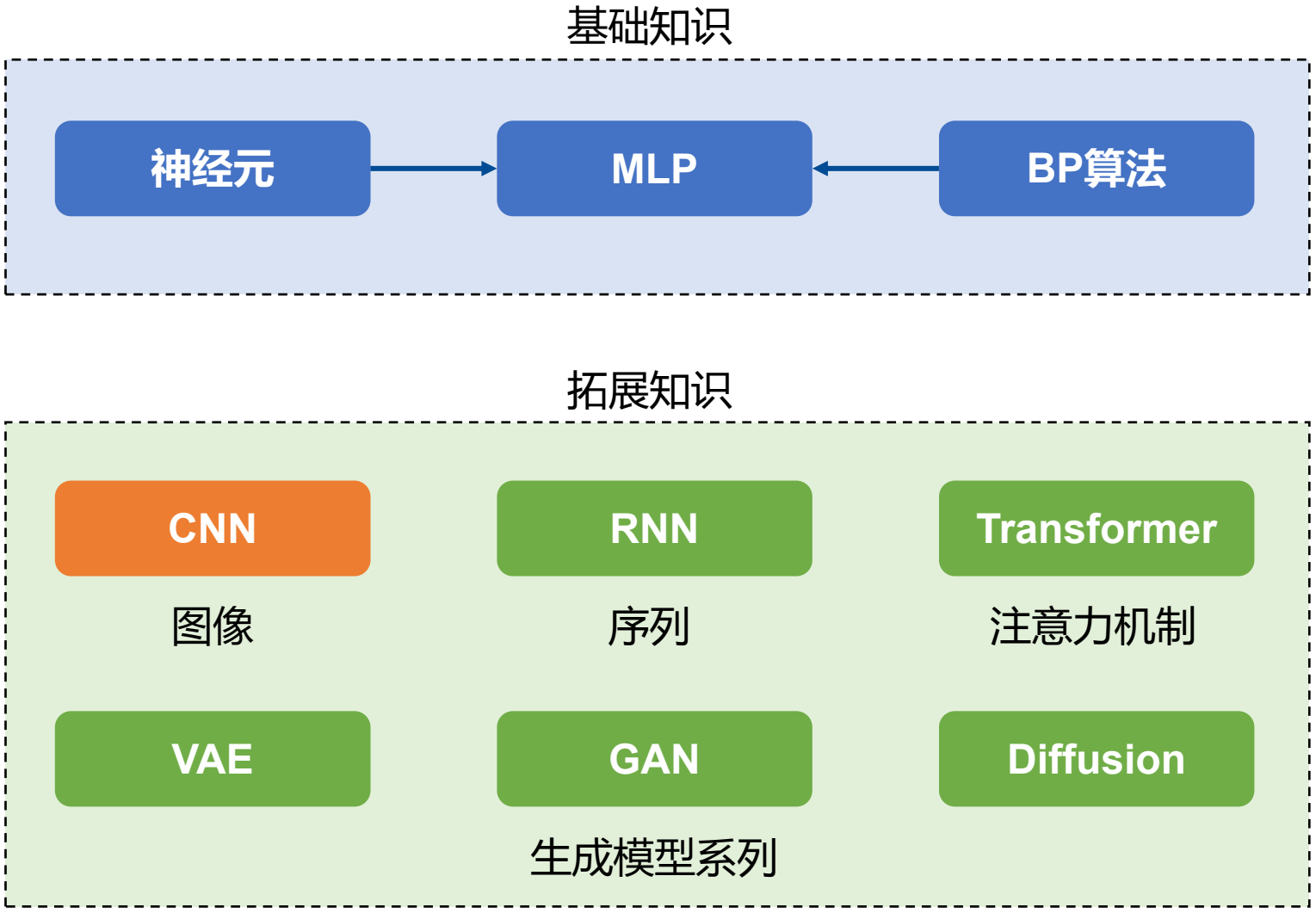
如果训练 50 个 epoch，表示：

- 整个数据集被完整看了 50 遍
- 参数总共更新了 $50 \times 10 = 500$ 次

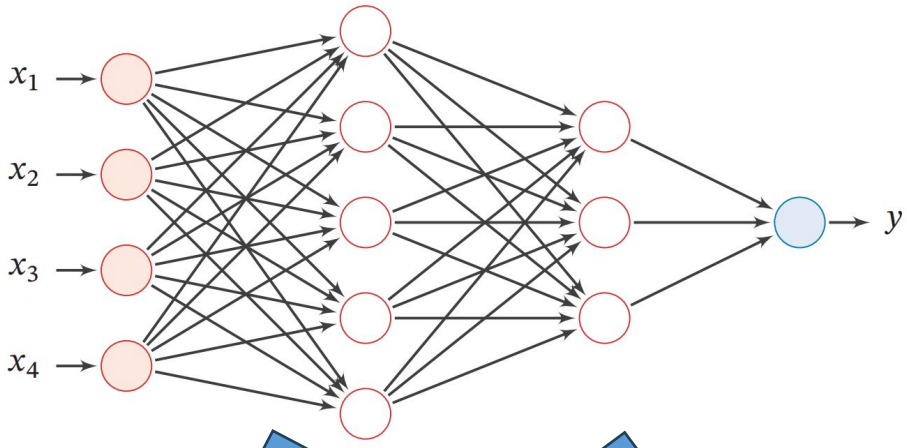
看看代码



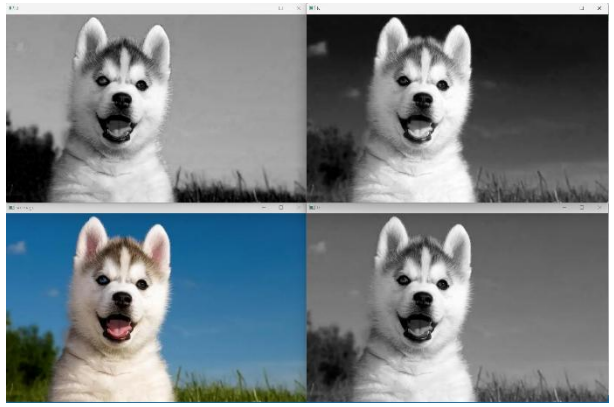
大纲


基础知识
神经元
MLP
BP算法
拓展知识
CNN
RNN
Transformer
图像
序列
注意力机制
VAE
GAN
Diffusion
生成模型系列

拓展



全连接前馈神经网络



卷积神经网络CNN

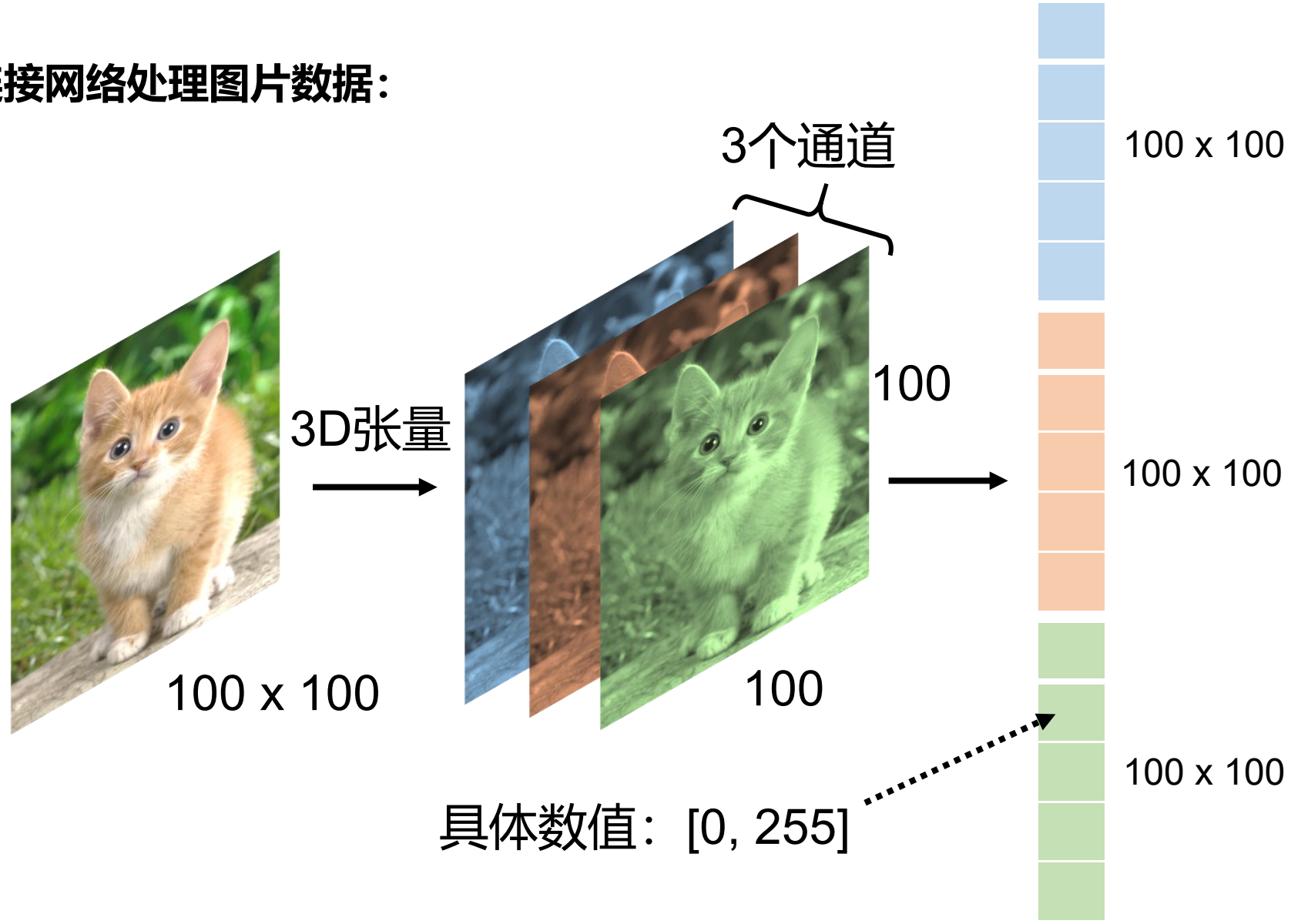
我爱学习

翻译

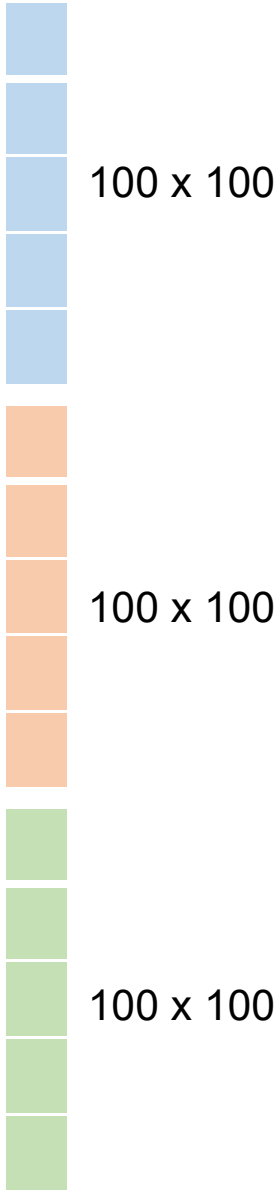
I love learning

循环神经网络RNN

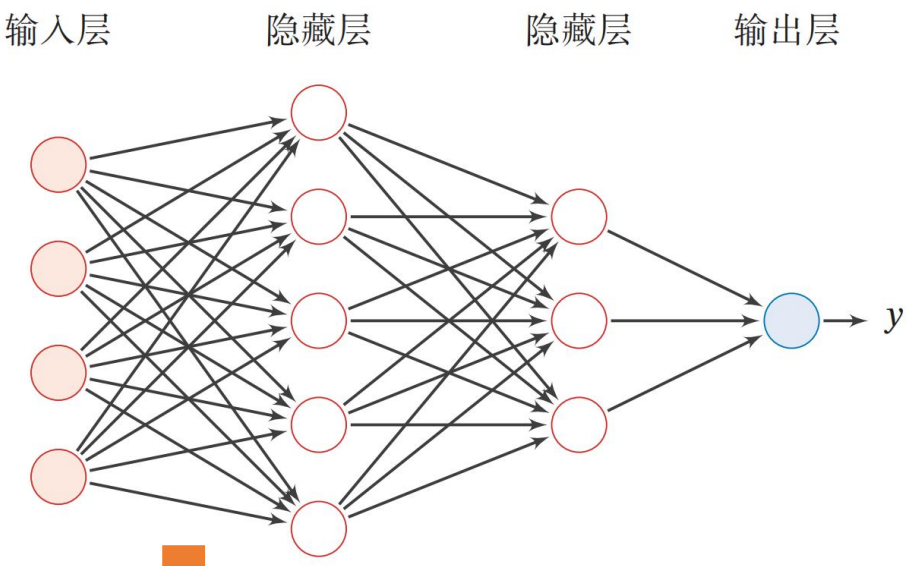
如果用全连接网络处理图片数据：



卷积神经网络(CNN)

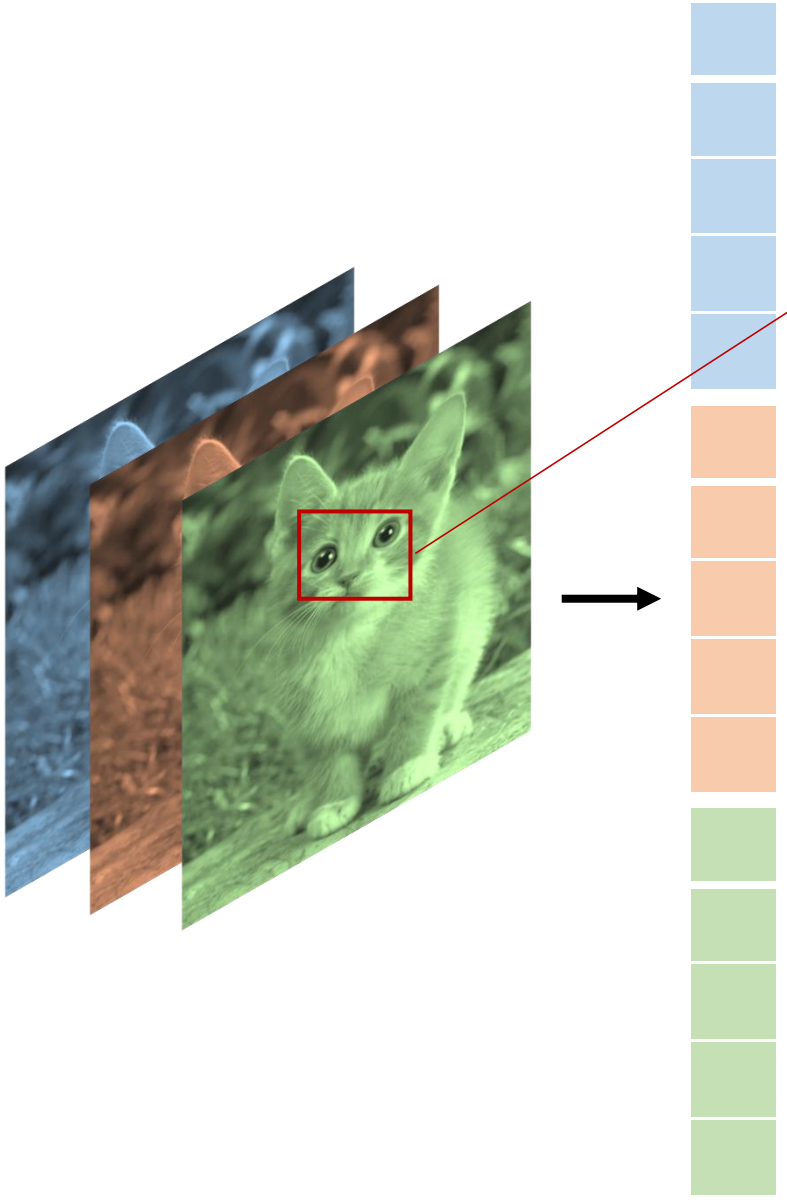


100 x 100 x 3　1000

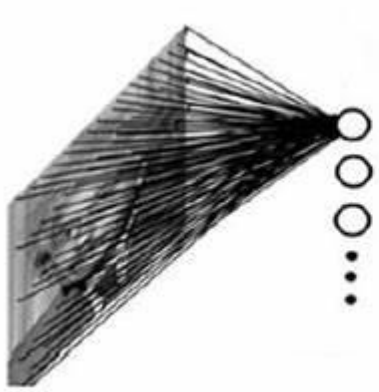


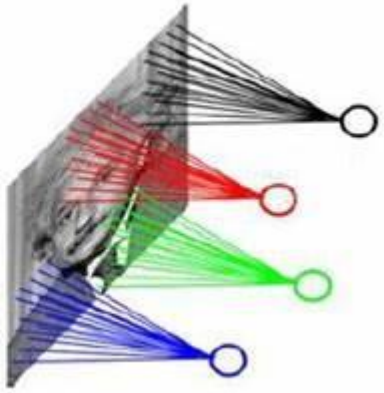
参数量： 3×10^7　太大了！

卷积神经网络(CNN)



应该保留图像的结构信息

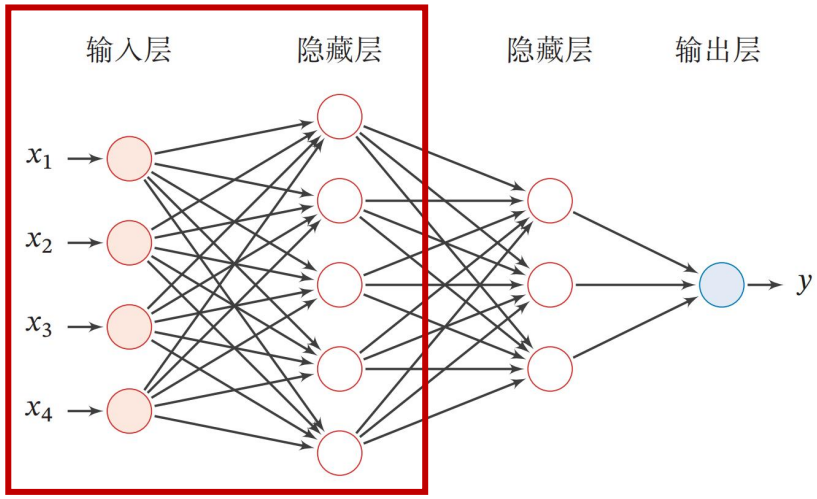


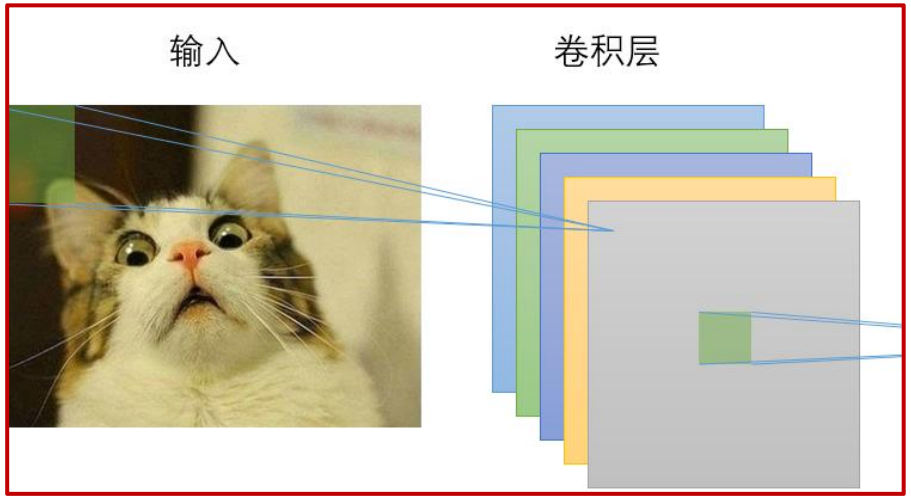


全局感受野

局部感受野

卷积神经网络(CNN)



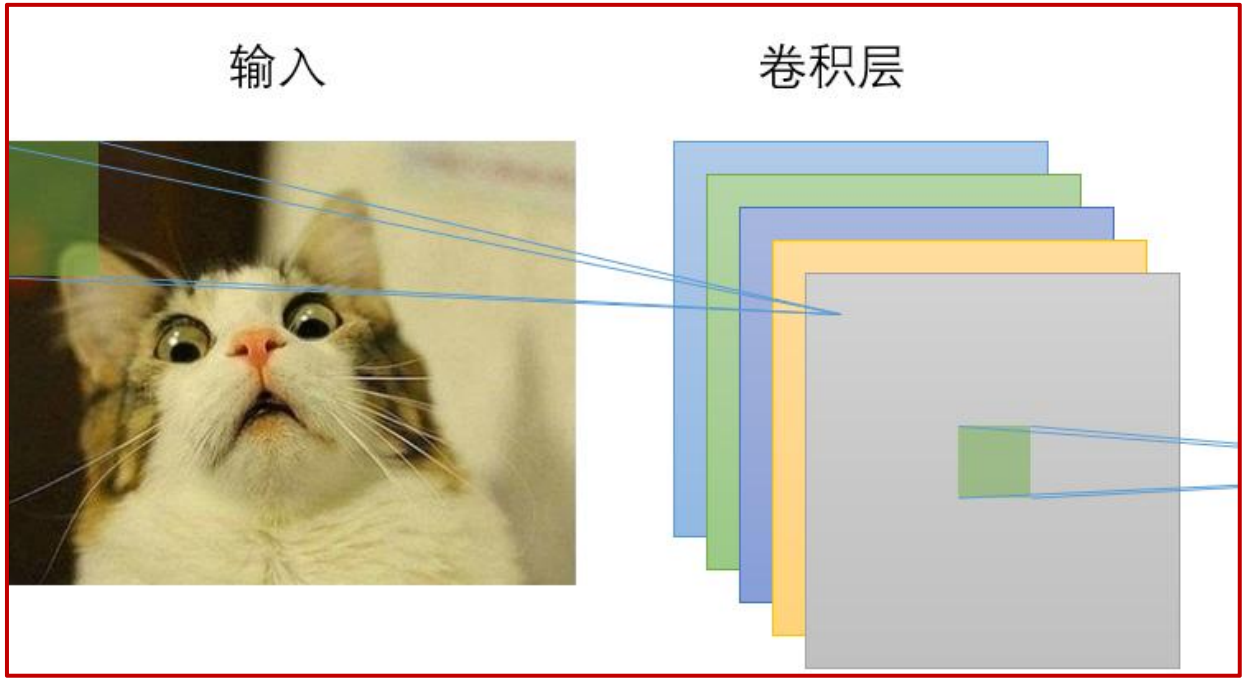


全连接层 → 卷积层

目标

- 保留图片全局结构
- 实现局部特征提取
- 减少参数量

卷积神经网络(CNN)



里面有什么?

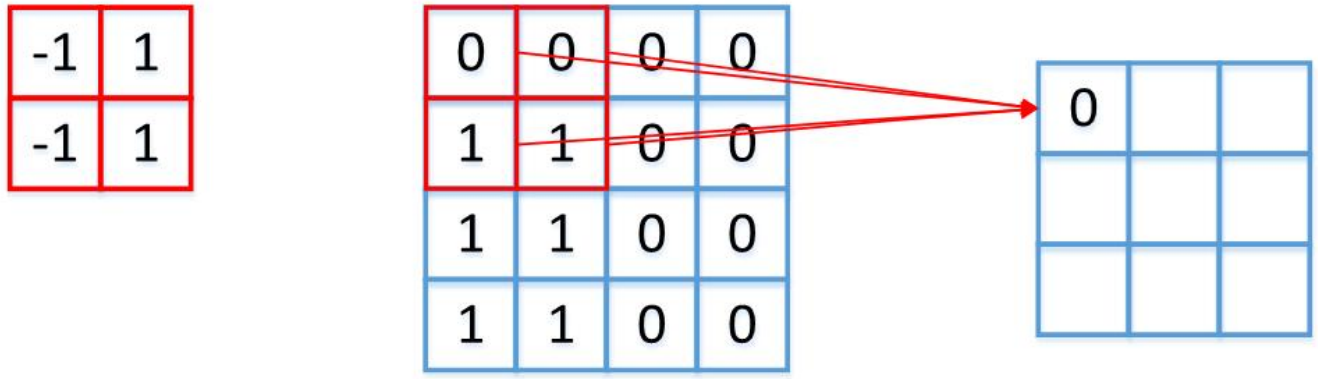


卷积神经网络(CNN)

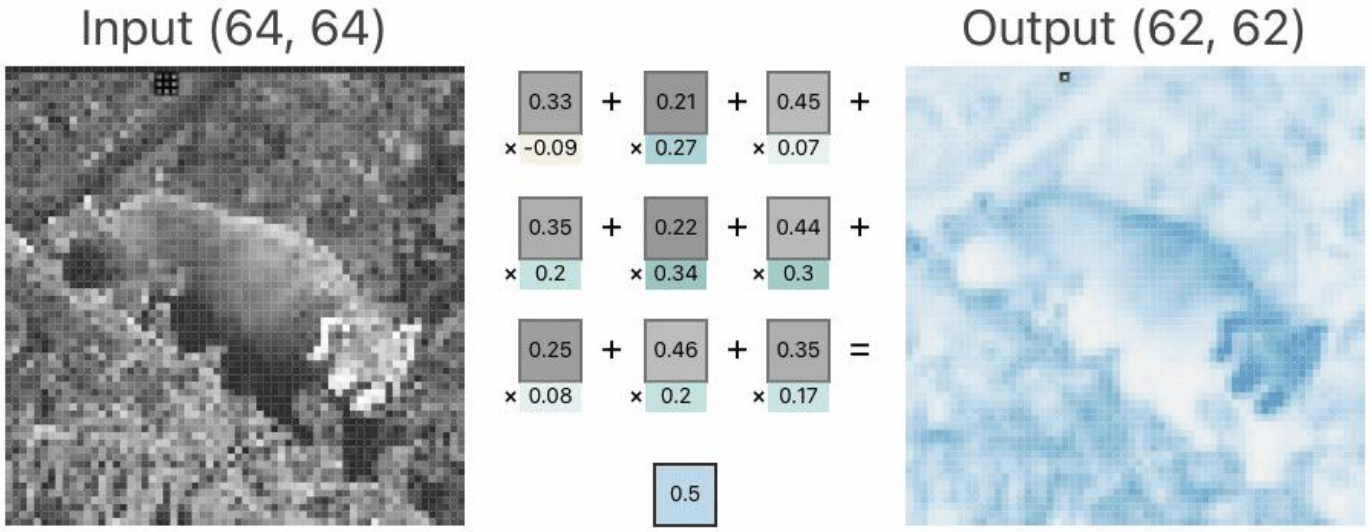
卷积核（滤波器）

+

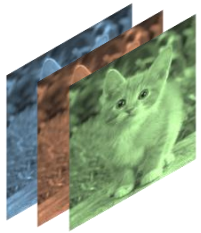
滑动窗口



卷积核(2x2)　　被卷积图片(4x4)　　卷积结果(3x3)



卷积神经网络(CNN)



3通道
卷积核

3	4	6	5	7
2	4	6	8	2
1	6	7	8	4
9	7	4	6	2
3	7	5	4	1

⊙

1	2	3
4	5	6
7	8	9

=

211	295	262
259	282	214
251	253	169

3	4	6	5	7
2	4	6	8	2
1	6	7	8	4
9	7	4	6	2
3	7	5	4	1

⊙

9	8	7
6	5	4
3	2	1

=

179	245	268
201	278	256
239	287	241

3	4	6	5	7
2	4	6	8	2
1	6	7	8	4
9	7	4	6	2
3	7	5	4	1

⊙

1	4	7
2	5	8
3	6	9

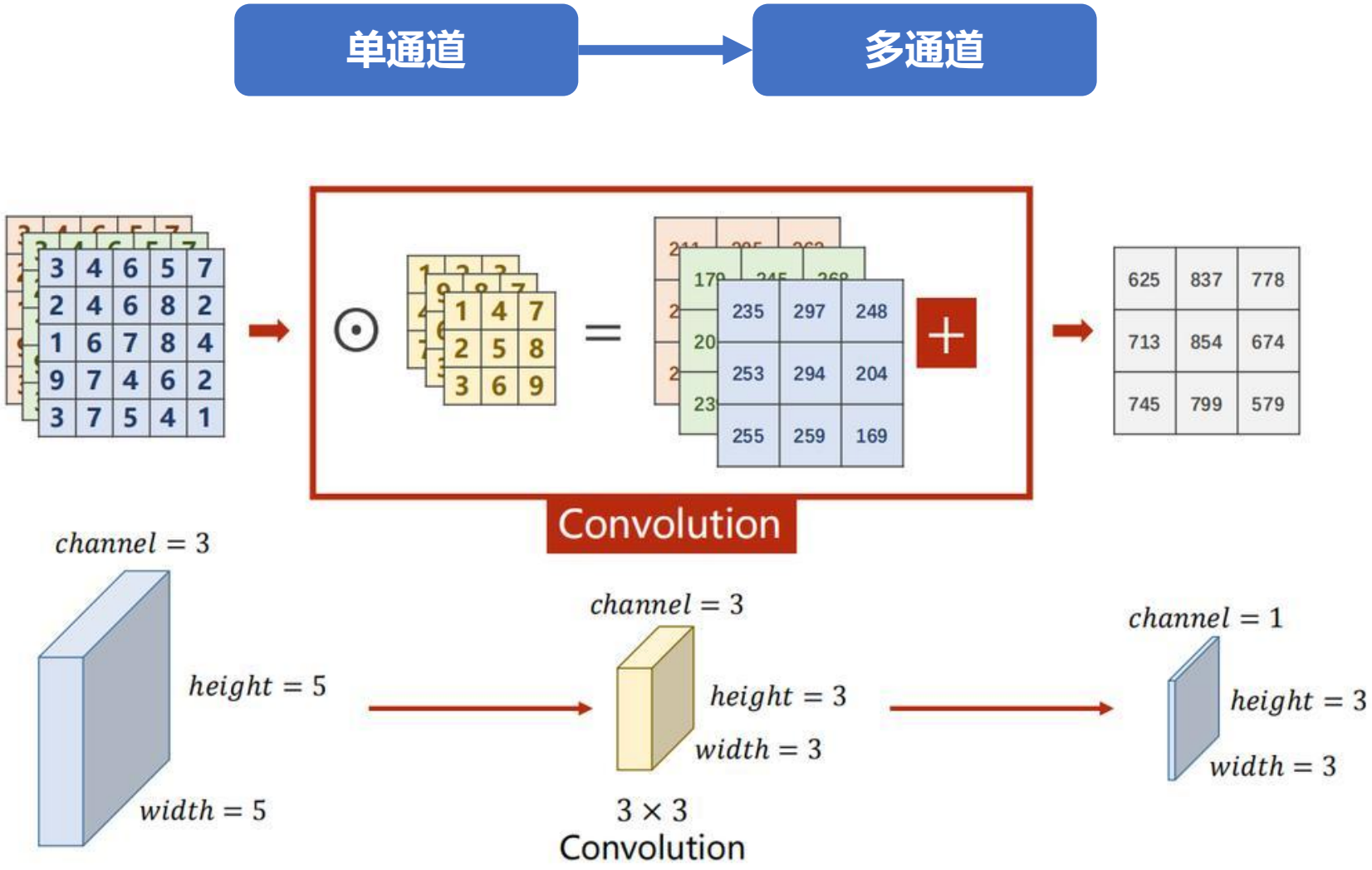
=

235	297	248
253	294	204
255	259	169

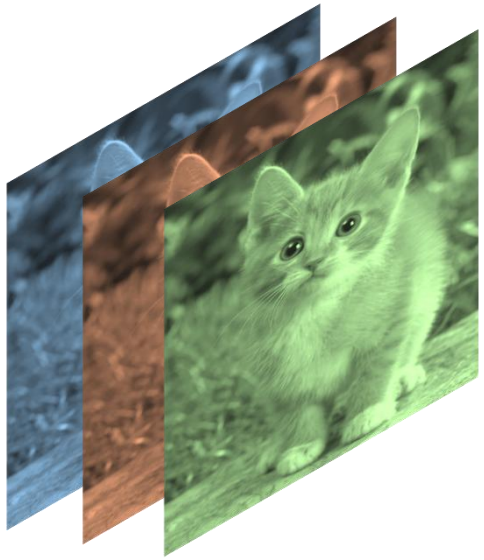
+

=

625	837	778
713	854	674
745	799	579


单通道
多通道
3 4 6 5 7
2 4 6 8 2
1 6 7 8 4
9 7 4 6 2
3 7 5 4 1
1 4 7
2 5 8
3 6 9
235 297 248
253 294 204
255 259 169
625 837 778
713 854 674
745 799 579
Convolution
channel = 3
height = 5
width = 5
channel = 3
height = 3
width = 3
3 × 3
Convolution
channel = 1
height = 3
width = 3

卷积神经网络(CNN)



3通道卷积核

滑动窗口



卷积是用来做什么的?



图片特征提取



不同卷积核可以提取不同图片特征

$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	

$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	
$\frac{1}{256}\begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$	

$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	

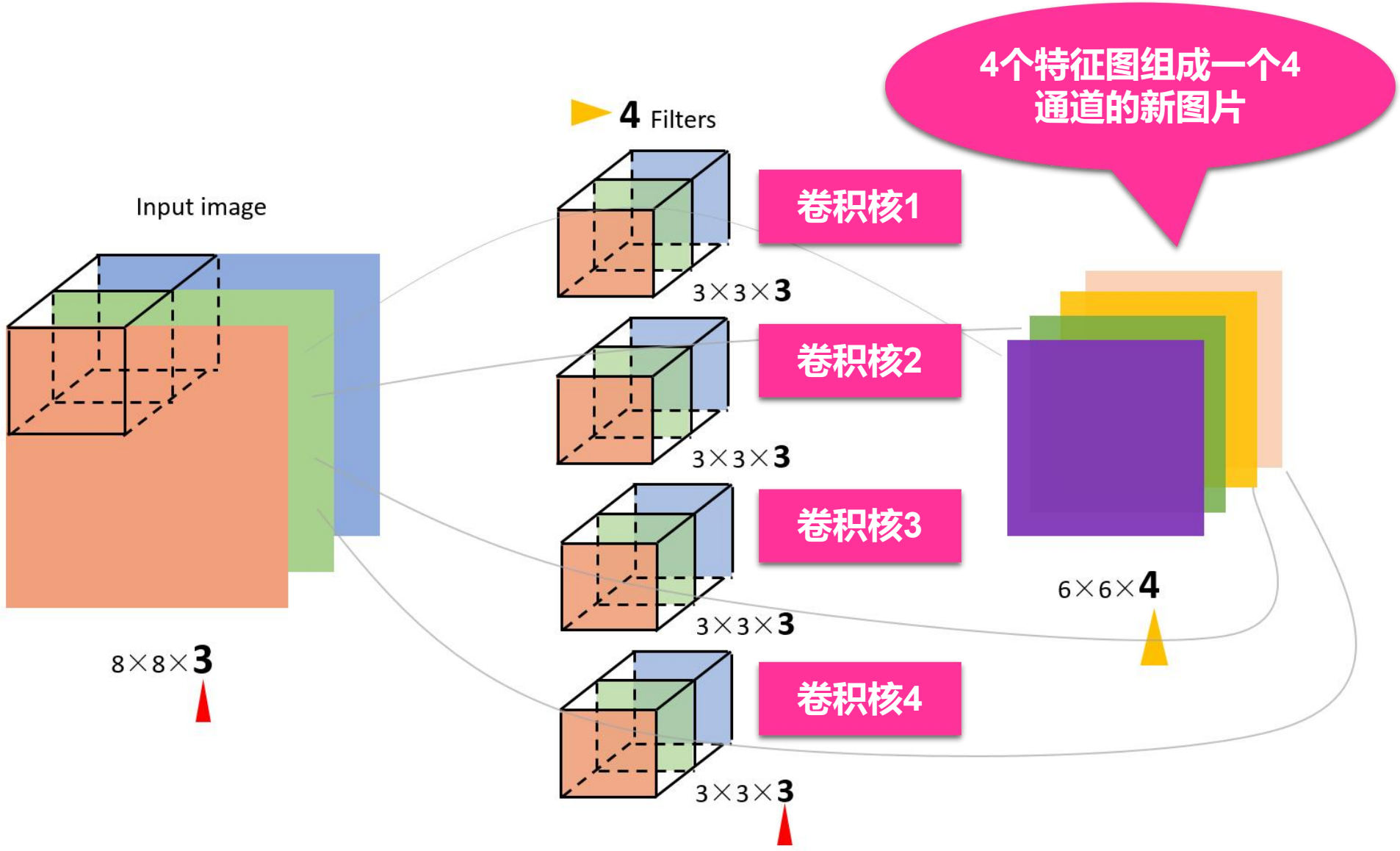


卷积核：待求解参数

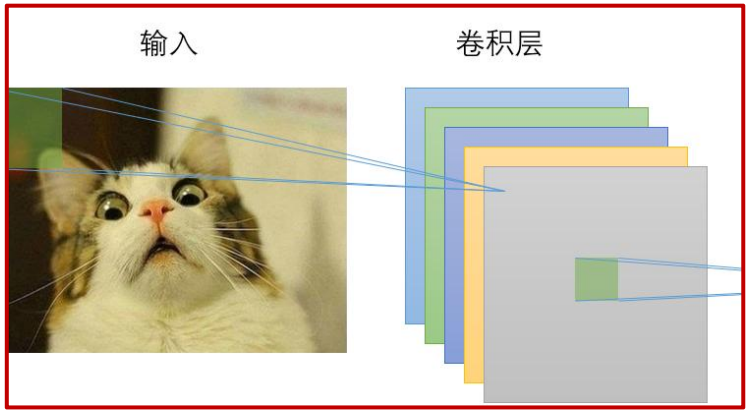
人工设计卷积核 → 让神经网络自己学习卷积核的参数

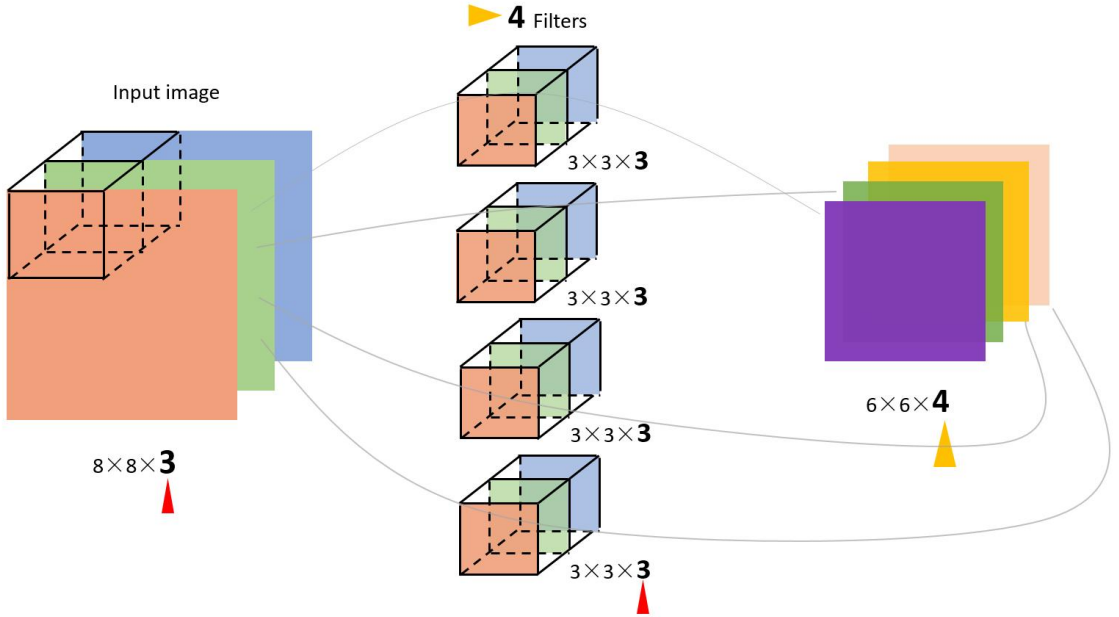
卷积神经网络(CNN)

多个卷积核



卷积神经网络(CNN)



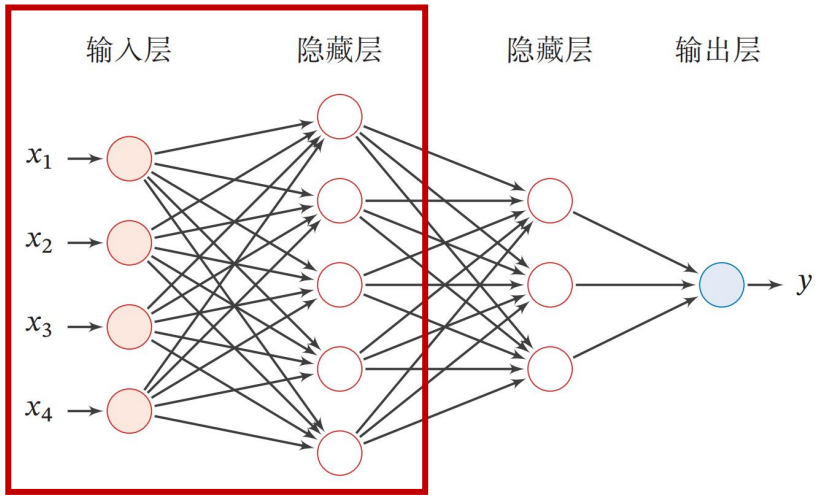


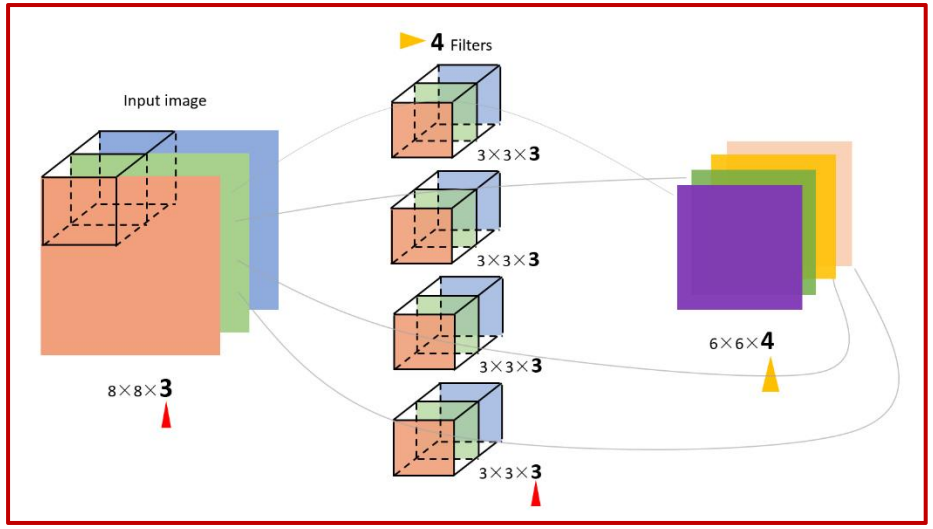
里面有什么?



- 卷积核
- 多个通道
- 滑动窗口
- 多个卷积核

卷积神经网络(CNN)





全连接层 → 卷积层

目标

保留图片全局结构

实现局部特征提取

减少参数量

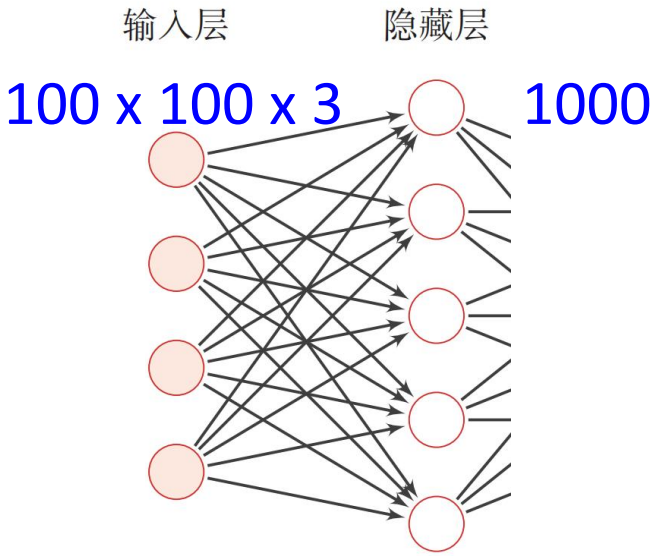
卷积核

多个通道

滑动窗口

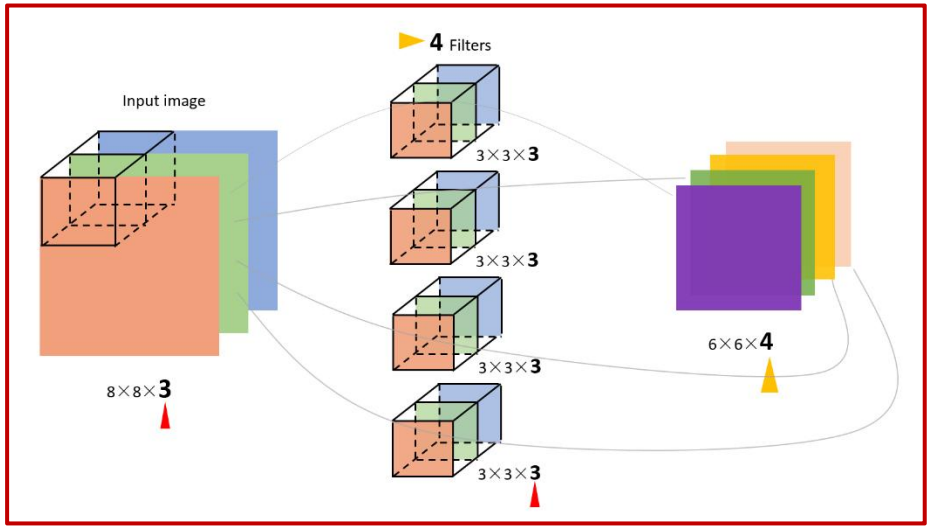
多个卷积核

全连接层



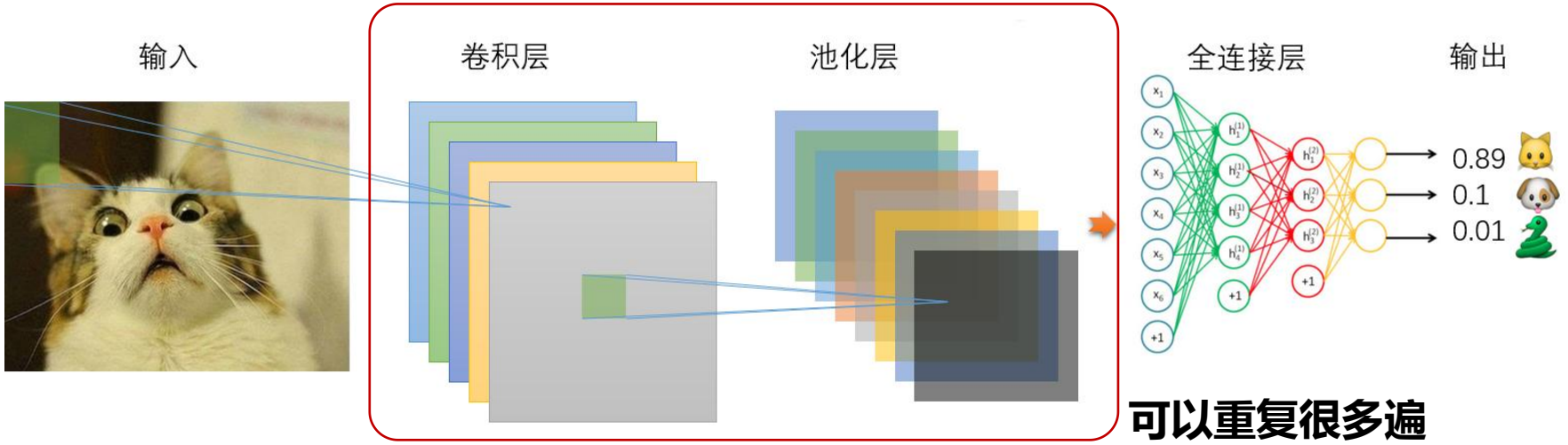
参数量： 3×10^7

卷积层



参数量： 27000

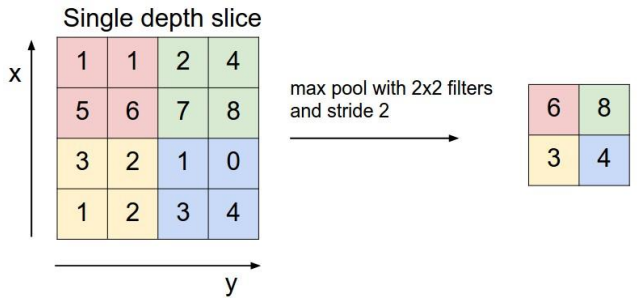
卷积神经网络(CNN)

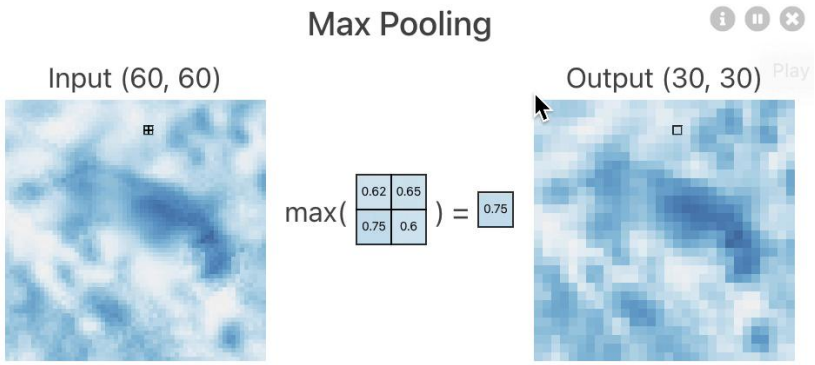


可以重复很多遍

或者 **卷积** **激活函数** **池化**

池化（降采样）





[ConvNetJS CIFAR-10 demo (stanford.edu)]

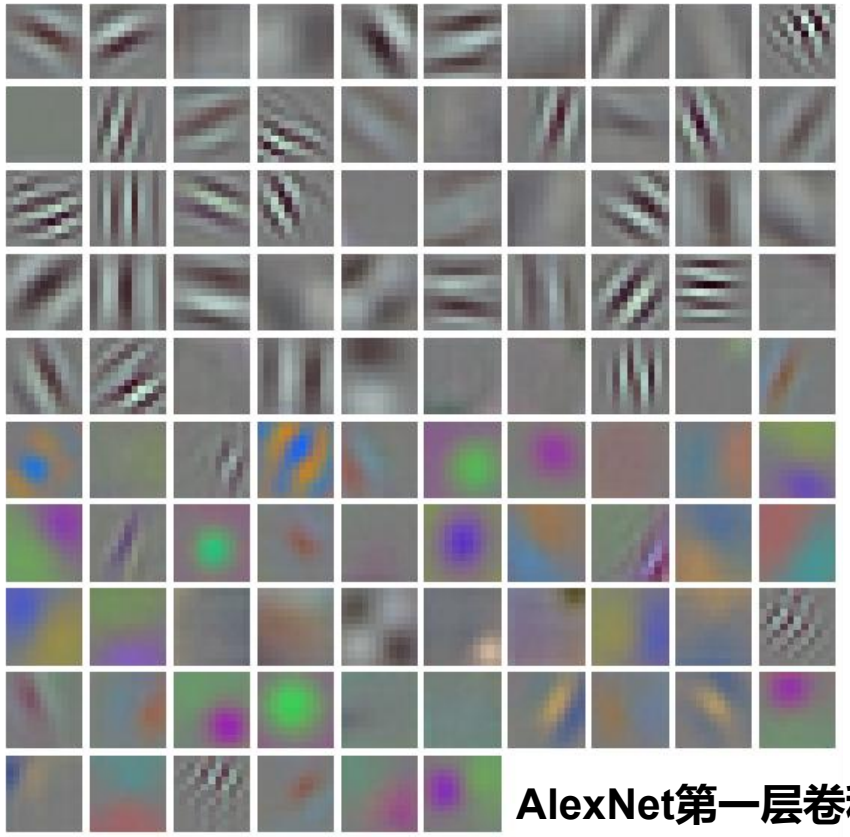
卷积神经网络(CNN)



CNN Explainer (poloclub.github.io)

卷积神经网络(CNN)

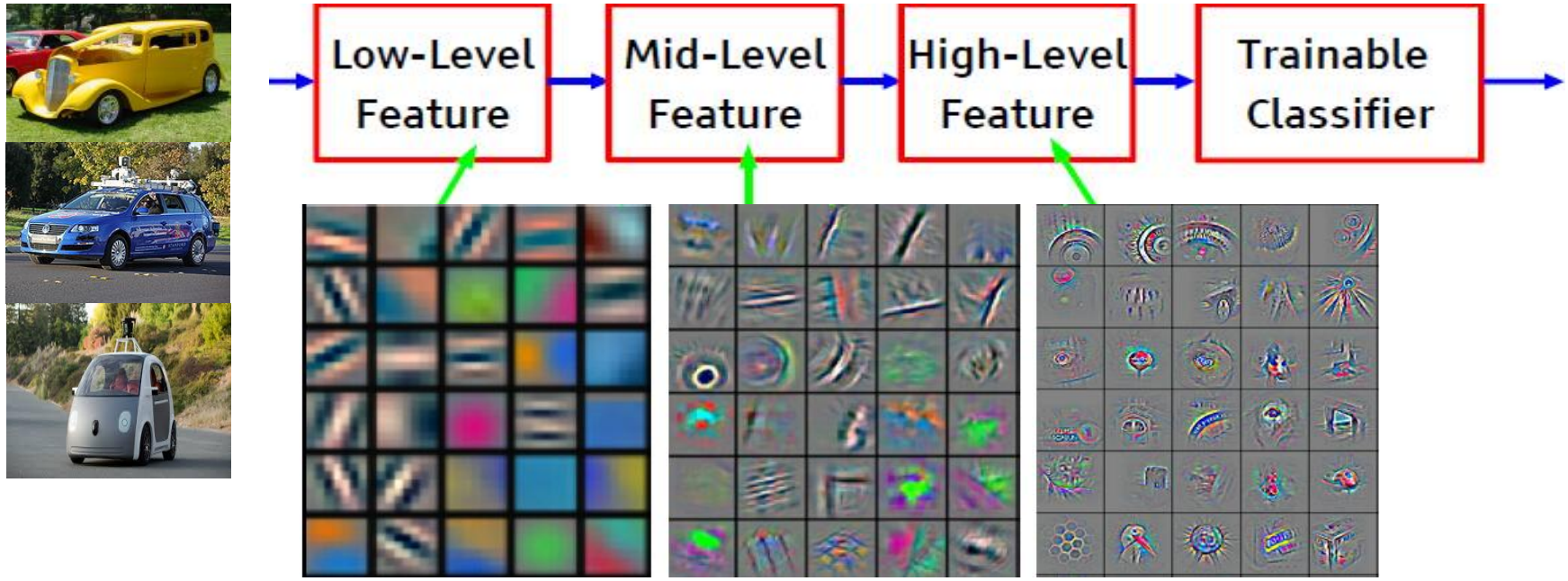
$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	

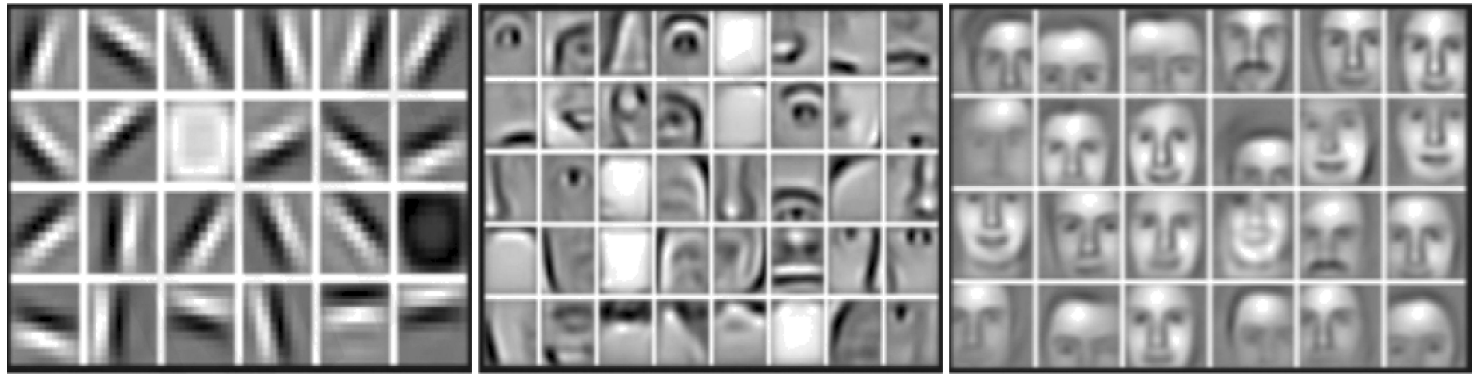


AlexNet第一层卷积核

人工设计卷积核 → 让神经网络自己学习卷积核的参数

卷积神经网络(CNN)

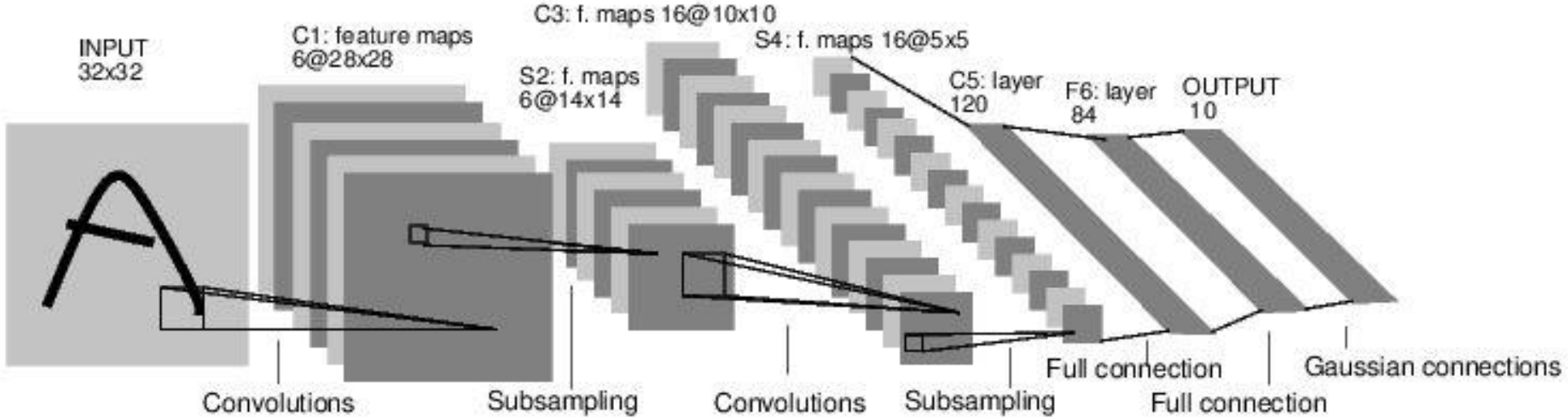




原始数据 → 底层特征 → 中层特征 → 高层特征 → 预测 → 结果

卷积神经网络(CNN)

LeNet
1980



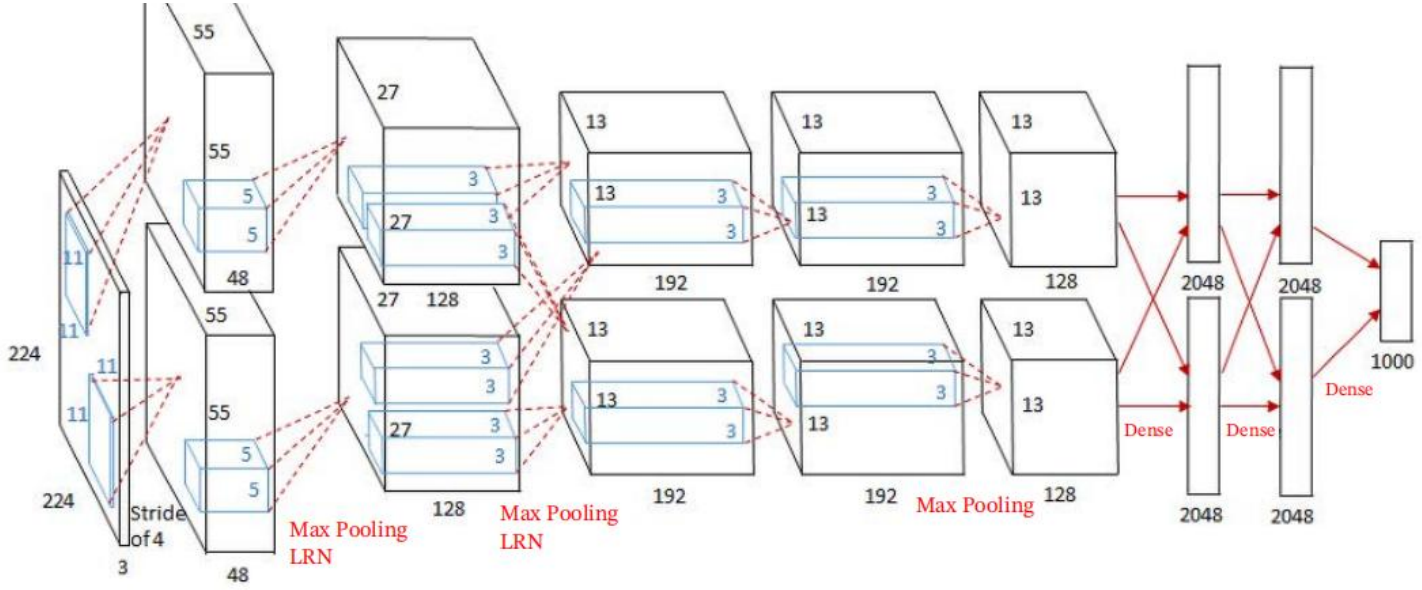


图 5.12　AlexNet 网络结构[1]

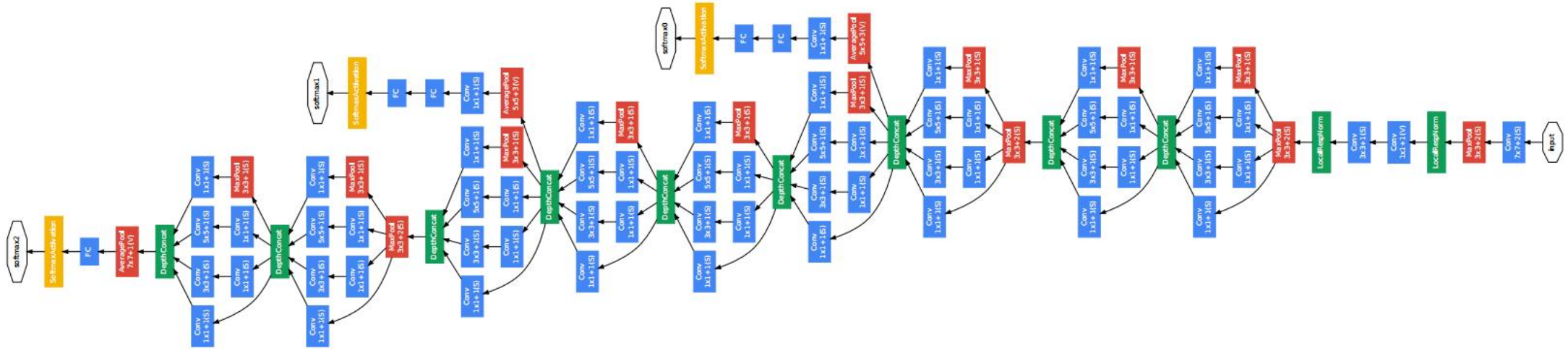
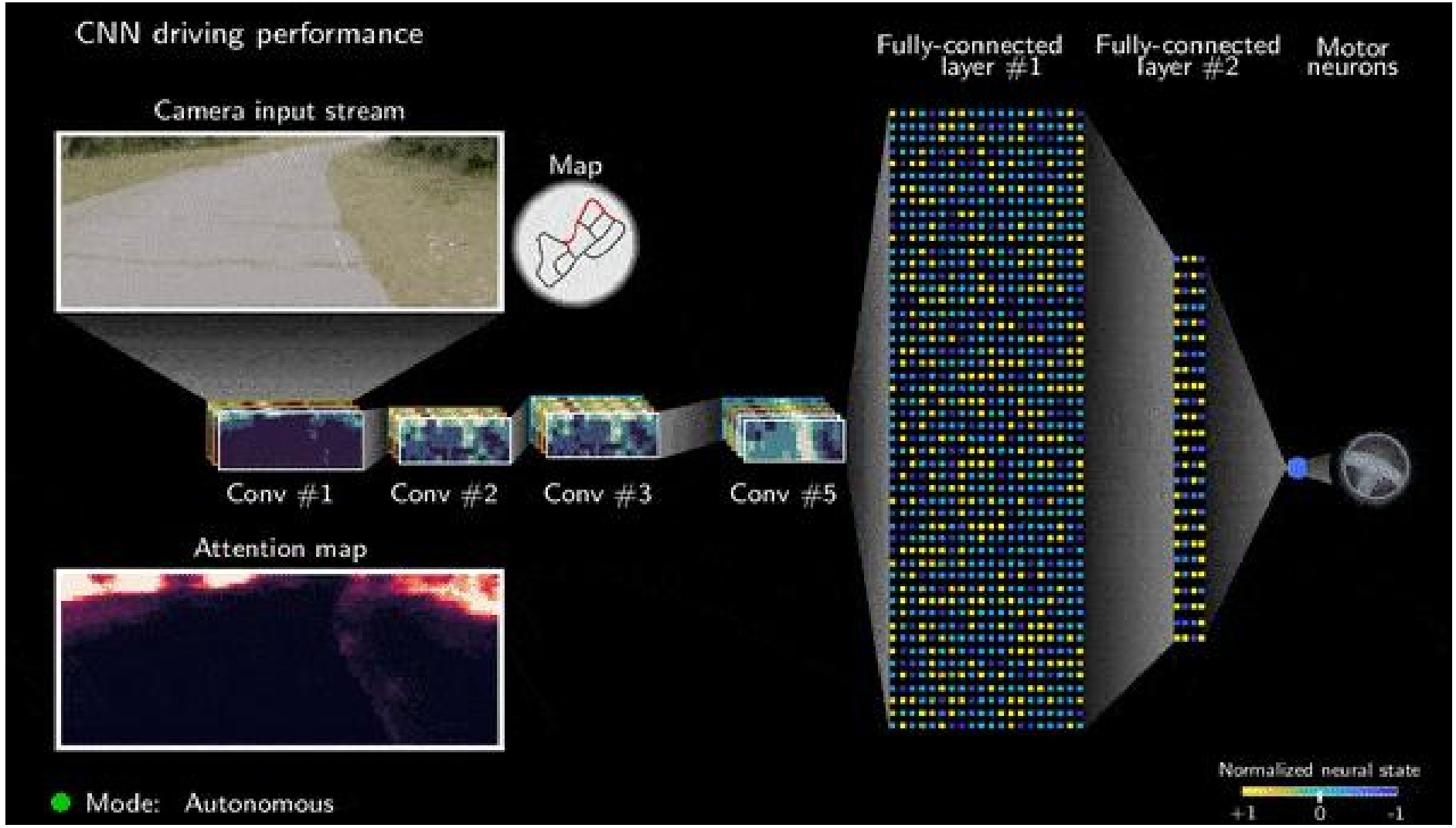


图 5.14　GoogLeNet 网络结构（图片来源：[Szegedy et al., 2015]）

卷积神经网络(CNN)


CNN driving performance
Camera input stream
Map
Fully-connected layer #1
Fully-connected layer #2
Motor neurons
Conv #1
Conv #2
Conv #3
Conv #5
Attention map
Normalized neural state
+1
0
-1
Mode: Autonomous

看看代码




谢 谢
首都师范大学
CAPITAL NORMAL UNIVERSITY